AETHER: AI-ENHANCED THREAT ENUMERATION AND RECONNAISSANCE PLATFORM

A PROJECT REPORT

Submitted by

MADHU KARTHICK A	**(111622108034)**
MURALI V	**(111622108040)**
SHYAM SUNDAR K	**(111622108054)**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

R. M. K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

ANNA UNIVERSITY: CHENNAI 600 025

DECEMBER 2025

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"AETHER: AI-ENHANCED THREAT ENUMERATION AND RECONNAISSANCE PLATFORM"** is the bonafide work of **MADHU KARTHICK A** (111622108034)**, MURALI V** (111622108040), **SHYAM SUNDAR K** (111622108054) who carried out the project work under my supervision.

SIGNATURE

DR. S. M. UDHAYA SANKAR

HEAD OF THE DEPARTMENT

Computer Science and Engineering (Cyber Security)

R.M.K College of Engineering and Technology

Puduvoyal - 601026

SIGNATURE

DR. DHARINI. N

SUPERVISOR

ASSOCIATE PROFESSOR

Computer Science and Engineering

(Cyber Security)

R.M.K College of Engineering and Technology

Puduvoyal - 601026

Submitted for the viva voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

Support on demand, encouragement at the needed moment and guidance in the right direction are indispensable for the success of any project. We have received these in excess from all corners from various people, and we are glad to submit our gratitude to them.

We thank **Thiru. R. S. Munirathinam**, our beloved Chairman, and **Thiru. R. M. Kishore**, our Vice Chairman of RMK Group of Institutions for extending a generous hand in providing the best of resources to the college.

We express our sincere gratitude to **Dr. N. Suresh Kumar**, the esteemed Head of our Institution, for his continuous motivation and guidance to all the faculty members and students. We also extend our heartfelt thanks to **Dr. K. Sivaram**, Dean–Academics, and **Dr. K. Ramar**, Dean–Research, for their constant support and encouragement throughout the course of this project.
Our sincere thanks to **Dr. S.M Udhaya Sankar**, the Head of the Department for his continuous support and motivation throughout our project.

We extend our profound gratitude to **Dr.Dharini N**, Associate Professor, our Project Coordinator and _____, our Supervisor for her guidance, who has indeed been a pollster throughout the course of the project, we thank them for giving us full support to complete the project successfully.

Last, but not the least, we take this opportunity to thank all the staff members of the Department of Computer Science and Engineering (Cyber Security). Regards to our family, classmates and friends who offered an unflinching moral support for completion of this project.

ABSTRACT

The project aims to enhance cybersecurity reconnaissance and attack surface understanding through the development of AETHER, an AI-assisted, legally compliant web intelligence platform. The system focuses on automated discovery, mapping, and analysis of web assets by integrating focused crawling techniques, graph-based visualization, and controlled execution of security reconnaissance tools. AETHER employs crawler-driven site exploration to construct structured sitemaps and relationship graphs that capture the interconnections between web pages, endpoints, and resources. These graph representations provide a visual and analytical foundation for understanding complex web infrastructures and potential exposure points.

To enrich reconnaissance depth, the platform orchestrates the execution of Kali Linux command-line tools within a sandboxed worker environment, ensuring safe, authorized, and resource-controlled analysis. Outputs from crawling and reconnaissance processes are normalized into structured data models and stored securely for further analysis. Building upon this collected evidence, AETHER incorporates a Retrieval-Augmented Generation (RAG) based conversational interface that enables users to query project-specific data and obtain contextual insights grounded in actual scan results. This AI-driven reasoning layer assists in summarizing findings, identifying patterns, and improving situational awareness without performing automated exploitation.

By combining focused crawling, graph-based intelligence, controlled automation, and AI-assisted reasoning, AETHER provides a comprehensive and scalable approach to external attack surface mapping. The platform emphasizes legal authorization, data integrity, and explainability, aiming to support security analysts, developers, and organizations in proactively understanding and reducing cyber risk. Through its modular design and visual-first analysis, AETHER contributes toward more transparent, interpretable, and responsible cybersecurity assessment practices.

Keywords: Cybersecurity reconnaissance, attack surface mapping, focused web crawling, graph visualization, Retrieval-Augmented Generation (RAG), AI-assisted security analysis

TABLE OF CONTENTS

CHAPTER NO.	TOPICS	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	ix
	LIST OF TABLES	ix
	LIST OF ABBREVIATIONS	x
1	**INTRODUCTION**	1
	1.1 Background of the Study	
	1.2 Domain Overview	
	1.3 Motivation of the Project	
	1.4 Problem Context	
	1.5 Need of the System / Gap Analysis	
	1.6 Scope of the Project	
	1.7 Limitations	
	1.8 Chapter Summary	
2	**LITERATURE REVIEW**	5
	2.1 Introduction to Literature Review	
	2.2 Existing Systems	
	2.3 Methods Used in Previous Studies	
	2.4 Comparative Analysis of Related Work	
	2.5 Limitations of Existing Research / Systems	
	2.6 Summary of Findings from Literature	
	2.7 Chapter Summary	

3	**PROBLEM STATEMENT**	10
	3.1 Statement of the Problem	
	3.2 Problem Description	
	3.3 Research / System Challenges	
	3.4 Proposed Solution Outline	
	3.5 Chapter Summary	
4	**OBJECTIVES**	13
	4.1 Primary Objectives	
	4.2 Secondary Objectives	
	4.3 Expected Outcome	
	4.4 Chapter Summary	
5	**PROPOSED SYSTEM**	15
	5.1 Overview of Proposed System	
	5.2 Block Diagram	
	5.3 System Architecture	
	5.4 Functional Description	
	5.5 Modules / Components Overview	
	5.6 Advantages of Proposed System	
	5.7 Chapter Summary	
6	**SYSTEM DESIGN**	21
	6.1 Introduction to System Design	
	6.2 UML Diagrams	

6.2.1 Use Case Diagram
6.2.2 Class Diagram
6.2.3 Sequence Diagram
6.2.4 Activity Diagram
6.2.5 Component Diagram
6.2.6 Deployment Diagram
6.3 Database Design
6.3.1 ER Diagram
6.3.2 Schema Diagram
6.3.3 Table Definitions
6.4 Hardware Requirements
6.5 Software Requirements
6.6 Chapter Summary

7 MODULE DESCRIPTION 29

7.1 User Management Module
7.1.1 Module Objectives
7.1.2 Module Overview
7.1.3 Input & Outputj Description
7.1.4 Algorithms / Logics Used
7.1.5 API / Functions Used
7.2 Project Management Module
7.2.1 Module Objectives
7.2.2 Module Overview
7.2.3 Input & Outputj Description
7.2.4 Algorithms / Logics Used
7.2.5 API / Functions Used
7.3 Reconnaissance Automation Module

7.3.1 Module Objectives
7.3.2 Module Overview
7.3.3 Input & Outputj Description
7.3.4 Algorithms / Logics Used
7.3.5 API / Functions Used
7.4 Web Crawling Module
7.4.1 Module Objectives
7.4.2 Module Overview
7.4.3 Input & Outputj Description
7.4.4 Algorithms / Logics Used
7.4.5 API / Functions Used
7.5 Mobile Security Analysis Module
7.5.1 Module Objectives
7.5.2 Module Overview
7.5.3 Input & Outputj Description
7.5.4 Algorithms / Logics Used
7.5.5 API / Functions Used
7.6 Graph Visualization Module
7.6.1 Module Objectives
7.6.2 Module Overview
7.6.3 Input & Outputj Description
7.6.4 Algorithms / Logics Used
7.6.5 API / Functions Used
7.7 AI Intelligence Module
7.7.1 Module Objectives
7.7.2 Module Overview
7.7.3 Input & Outputj Description
7.7.4 Algorithms / Logics Used
7.7.5 API / Functions Used

7.8 Job Queue & Worker Module
7.8.1 Module Objectives
7.8.2 Module Overview
7.8.3 Input & Outputj Description
7.8.4 Algorithms / Logics Used
7.8.5 API / Functions Used
7.9 Data Storage Module
7.9.1 Module Objectives
7.9.2 Module Overview
7.9.3 Input & Outputj Description
7.9.4 Algorithms / Logics Used
7.9.5 API / Functions Used
7.10 Chapter Summary

8 IMPLEMENTATION 37

8.1 Implementation Details
8.2 Technology Stack
8.3 System Architecture Implementation
8.4 Backend Implementation
8.5 Frontend Implementation
8.6 Database Implementation
8.7 Job Queue Implementation
8.8 Reconnaissance Tool Implementation
8.9 AI Intelligence Implementation
8.10 Graph Visualization Implementation
8.11 Security Implementation
8.12 API Implementation
8.13 Deployment Implementation

8.14 Testing Implementation
8.15 Implementation Challenges
8.16 Chapter Summary

9 RESULTS AND DISCUSSIONS 42

9.1 Introduction
9.2 System Execution Results
9.3 Reconnaissance Results
9.4 Web Crawling Analysis Results
9.5 Mobile Application Analysis Results
9.6 AI Intelligence Results
9.7 Visualization Results
9.8 Performance Evaluation
9.9 Usability Analysis
9.10 Comparative Discussion
9.11 Limitations Observed
9.12 Chapter Summary

10 CONCLUSION 47

REFERENCES 48

APPENDIX 51

A. Configuration and Environment Details
B. Supported Reconnaissance Tools
C. AI and RAG Configuration
D. Security and Legal Safeguards

LIST OF FIGURES

Figure No.	Title / Description	Page No.
5.1	Flow Map / Block Diagram	16
5.2	System Architecture	17
5.3	Modular Architecture View	18
6.1	Sequence Diagram	22
6.2	Activity Diagram	23
6.3	ER Diagram	25
9.1	Malware Report	42
9.2	Web Recon	43
9.3	RAG LLM	45

LIST OF TABLES

TABLE NO:	TITLE	PAGE NO:
9.1	Comparison of Existing Solution	44

LIST OF ABBREVIATIONS

Acronym	Full Form
AETHER	Automated Ethical Threat Exploration and Reconnaissance
AI	Artificial Intelligence
API	Application Programming Interface
ASM	Attack Surface Management
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
DB	Database
ER	Entity Relationship
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IAM	Identity and Access Management
JSON	JavaScript Object Notation
LLM	Large Language Model
OSINT	Open Source Intelligence
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFC	Request for Comments
SIEM	Security Information and Event Management
SQL	Structured Query Language
SSH	Secure Shell
UI	User Interface
URL	Uniform Resource Locator
VM	Virtual Machine
VPS	Virtual Private Server
XSS	Cross-Site Scripting

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND OF THE STUDY

With the rapid expansion of the internet and cloud-based infrastructures, organizations increasingly rely on complex web applications, APIs, and distributed services to deliver functionality. While this digital transformation has enabled scalability and accessibility, it has also significantly expanded the external attack surface exposed to adversaries. Modern web environments consist of dynamic pages, third-party integrations, client-side scripts, hidden endpoints, and legacy components that are often poorly documented or overlooked.

Traditional security assessment practices primarily depend on manual reconnaissance or standalone automated scanners, which frequently operate in isolation and provide fragmented outputs. These methods often lack a unified view of discovered assets and fail to represent the interrelationships between web components in a comprehensible manner. As a result, security teams face challenges in understanding the true extent of their attack surface and prioritizing risks effectively.

Recent advances in automation, graph-based modeling, and artificial intelligence present opportunities to improve how reconnaissance data is collected, visualized, and interpreted. The AETHER project emerges within this context, aiming to provide a structured, visual, and AI-assisted approach to authorized web reconnaissance and attack surface mapping.

1.2 DOMAIN OVERVIEW

The domain of this project lies at the intersection of cybersecurity, web intelligence, and artificial intelligence. Specifically, AETHER operates within the following subdomains:

- **Web Security and Reconnaissance:** Identification of publicly accessible assets, endpoints, technologies, and configurations.

- **Focused Web Crawling:** Controlled exploration of websites to discover relevant pages and resources while minimizing noise and legal risk.
- **Graph-Based Security Analysis:** Representation of web assets and their relationships using node-edge models to enhance interpretability.
- **Security Automation:** Orchestration of reconnaissance tools for efficient and repeatable data collection.
- **AI-Assisted Reasoning (RAG):** Application of Retrieval-Augmented Generation to analyze and query collected security data contextually.

By combining these domains, AETHER aims to bridge the gap between raw reconnaissance outputs and actionable security understanding.

1.3 MOTIVATION OF THE PROJECT

The primary motivation behind AETHER is the lack of holistic visibility in existing reconnaissance workflows. Security practitioners often rely on multiple tools such as scanners, crawlers, and OSINT utilities, each producing disjointed outputs that require manual correlation. This process is time-consuming, error-prone, and difficult to scale.

Additionally, many modern platforms do not adequately address:

- Legal and ethical constraints of reconnaissance
- Visualization of attack surfaces for non-expert stakeholders
- Contextual reasoning over large volumes of scan data

AETHER is motivated by the need for a system that not only automates reconnaissance but also organizes, visualizes, and explains findings in a safe and intelligible manner. The integration of AI-based reasoning further motivates the project by enabling natural-language interaction with complex security data.

1.4 PROBLEM CONTEXT

Organizations frequently lack a clear and up-to-date understanding of their exposed web assets. Factors contributing to this problem include rapid development cycles, third-party services, shadow IT, and undocumented endpoints. Conventional tools generate extensive logs and reports but provide limited insight into how different components relate to one another or contribute to overall risk.

Furthermore, executing reconnaissance tools without proper isolation or authorization can lead to legal, ethical, and operational issues. Serverless platforms are unsuitable for heavy scanning tasks, while ad-hoc scripts lack standardization and safety controls.

The problem addressed by AETHER is the absence of a secure, scalable, and interpretable platform that combines focused crawling, automated reconnaissance, structured storage, and intelligent analysis under a unified architecture.

1.5 NEED OF THE SYSTEM/GAP ANALYSIS

Despite the availability of numerous security tools, several gaps remain:

- Fragmented Tooling: Existing scanners operate independently and do not provide a consolidated view.
- Poor Visualization: Limited support for graph-based representation of web structures and relationships.
- Lack of Contextual Intelligence: Raw outputs are difficult to interpret without expert knowledge.
- Scalability Constraints: Serverless platforms cannot support long-running reconnaissance tasks.
- Legal and Safety Oversight: Insufficient enforcement of authorization, rate limits, and target validation.

AETHER addresses these gaps by introducing a system that integrates crawling, scanning, visualization, and AI-assisted reasoning while enforcing strict safety and legal controls.

1.6 SCOPE OF THE PROJECT

The scope of AETHER includes:

- Authorized external web reconnaissance
- Focused crawling and sitemap generation
- Graph-based visualization of web assets and relationships
- Controlled execution of reconnaissance tools in a sandboxed environment
- Secure storage and retrieval of scan results
- AI-assisted querying and summarization using RAG techniques

The project does not include automated exploitation, payload delivery, or offensive attack execution. Its purpose is limited to analysis, visualization, and intelligence generation for defensive and educational use cases.

1.7 LIMITATIONS

While AETHER offers a comprehensive approach, certain limitations exist:

- Dependence on authorization restricts applicability to permitted targets only
- Crawling depth and scan coverage are limited by resource and time constraints
- AI-generated insights depend on the quality and completeness of collected data
- The system does not replace expert judgment or manual penetration testing
- RAG-based responses may require continuous tuning to avoid misinterpretation

These limitations highlight areas for future enhancement rather than deficiencies in system design.

1.8 CHAPTER SUMMARY

This chapter introduced the background, motivation, and problem context behind the AETHER project. It outlined the cybersecurity domain in which the system operates and identified critical gaps in existing reconnaissance approaches. The need for a unified, visual, and AI-assisted platform was established, along with the scope and limitations of the proposed system. Subsequent chapters will detail the system architecture, methodologies, implementation, and evaluation of AETHER.

CHAPTER 2
LITERATURE REVIEW

2.1 INTRODUCTION TO LITERATURE REVIEW

A literature review provides a critical examination of existing research, technologies, and methodologies relevant to the proposed system. In the domain of cybersecurity reconnaissance and attack surface analysis, several studies have explored web crawling, vulnerability scanning, attack graph modeling, OSINT automation, and AI-assisted security analysis. However, most of these approaches remain fragmented, tool-centric, or theoretical in nature.

This chapter reviews prominent research works and existing systems related to web reconnaissance, security automation, graph-based analysis, and AI-driven cybersecurity intelligence. The limitations of these studies are identified, and the ways in which the proposed system, AETHER, addresses these limitations are clearly articulated.

2.2 EXISTING SYSTEMS

Paper 1: GAT-APG – Graph Attention Network-Based Attack Path Generation [1]

Description: This paper proposes a Graph Attention Network (GAT) model to generate attack paths for security simulation by analyzing relationships between vulnerabilities and system components.

Limitation: The approach relies on predefined system graphs and is primarily simulation-based, lacking real-time reconnaissance and early-stage threat enumeration.

Solution Provided by AETHER: AETHER dynamically constructs threat graphs using live reconnaissance data, enabling real-time threat enumeration before attack path prioritization.

Paper 2: SPGNN-API – Transferable Graph Neural Network for Attack Path Identification [2]

Description: This work introduces a transferable GNN model to identify attack paths and support autonomous mitigation across different environments.

Limitation: The system assumes the availability of structured attack graphs and focuses mainly on attack path identification rather than holistic security assessment.

Solution Provided by AETHER: AETHER automates the generation of threat graphs directly from reconnaissance outputs, removing dependency on pre-existing structured graphs.

Paper 3: Cyber-Security Knowledge Graph Generation by Hierarchical Nonnegative Matrix Factorization [3]

Description: This study presents a method for generating cybersecurity knowledge graphs using hierarchical nonnegative matrix factorization techniques.

Limitation: The approach focuses on knowledge representation and does not integrate live reconnaissance or vulnerability enumeration mechanisms.

Solution Provided by AETHER: AETHER integrates knowledge-based reasoning with continuous reconnaissance and enumeration to maintain up-to-date security intelligence.

Paper 4: AIPenTool – A Unified Approach to Automated Penetration Testing [4]

Description: This paper proposes an automated penetration testing framework aimed at enhancing network and web application security.

Limitation: The framework relies heavily on predefined testing logic and lacks adaptive intelligence for dynamic attack surfaces.

Solution Provided by AETHER: AETHER emphasizes AI-assisted threat enumeration and contextual prioritization rather than static exploitation workflows.

Paper 5: Automated Penetration Testing Tool [5]

Description: This work presents an automated tool for penetration testing that reduces manual effort through scripted vulnerability detection.

Limitation: The tool is rule-based and does not adapt to unknown vulnerabilities or changing system behavior.

Solution Provided by AETHER: AETHER uses AI-driven analysis to prioritize threats dynamically based on reconnaissance data.

Paper 6: Automated Security Penetration Testing Based on Machine Learning [6]

Description: This paper applies machine learning techniques to automate penetration testing decisions and improve detection accuracy.

Limitation: The approach requires extensive training data and focuses primarily on exploitation rather than reconnaissance.

Solution Provided by AETHER: AETHER focuses on early-stage reconnaissance and enumeration, reducing dependence on large labeled datasets.

Paper 7: Automated Penetration Testing Architecture Using Metasploit and OWASP ZAP [7]

Description: This study integrates Metasploit and OWASP ZAP to automate web application penetration testing.

Limitation: The architecture depends on existing tools and produces large volumes of uncorrelated vulnerability data.

Solution Provided by AETHER: AETHER correlates reconnaissance outputs into structured threat models to reduce alert overload.

Paper 8: Automated Penetration Testing Based on Adversarial Inverse Reinforcement Learning [8]

Description: This work applies adversarial inverse reinforcement learning to enable penetration testing systems to learn attack strategies.

Limitation: The model is complex, computationally intensive, and focuses mainly on exploitation strategies.

Solution Provided by AETHER: AETHER applies lightweight AI techniques for prioritization and reasoning rather than exploitation learning.

Paper 9: Automated Penetration Testing – A Systematic Review [9]

Description: This paper provides a comprehensive review of automated penetration testing techniques and tools.

Limitation: The review highlights fragmentation and lack of integration among existing solutions.

Solution Provided by AETHER: AETHER provides a unified platform integrating reconnaissance, enumeration, and prioritization.

Paper 10: Open Source Solutions for Vulnerability Assessment [10]

Description: This study compares open-source vulnerability assessment tools used in cybersecurity.

Limitation: The tools operate independently and lack intelligent prioritization mechanisms.

Solution Provided by AETHER: AETHER aggregates findings into a unified threat intelligence model.

Paper 11: Scorpio – Automated Penetration Testing Tool [11]

Description: This paper presents an automated penetration testing tool integrated with a cyber range.

Limitation: The tool is primarily designed for training environments and lacks real-world adaptability.

Solution Provided by AETHER: AETHER is designed for real-world deployment with dynamic reconnaissance capabilities.

Paper 12: SoURCERER – Developer-Driven Security Testing Framework [12]

Description: This framework focuses on developer-driven security testing for Android applications.

Limitation: It is platform-specific and limited to application-level testing.

Solution Provided by AETHER: AETHER adopts a platform-agnostic approach applicable to networks and web applications.

Paper 13: Parallelized Cyber Reconnaissance Automation [13]

Description: This study proposes a real-time and scheduled cyber reconnaissance automation framework.

Limitation: The approach lacks intelligent analysis and prioritization of reconnaissance results.

Solution Provided by AETHER: AETHER enhances reconnaissance outputs with AI-driven threat enumeration and prioritization.

Paper 14: webFuzz – Grey-Box Fuzzing for Web Applications [14]

Description: This paper introduces a grey-box fuzzing technique for web applications.

Limitation: The approach focuses on input fuzzing and does not provide holistic system-level threat analysis.

Solution Provided by AETHER: AETHER integrates fuzzing outputs into a broader threat enumeration framework.

Paper 15: B2SMatcher – Fine-Grained Version Identification [15]

Description: This work focuses on identifying open-source software versions in binary files.

Limitation: The approach is limited to software identification and does not assess exploitability.

Solution Provided by AETHER: AETHER correlates version identification results with known vulnerabilities and attack surfaces.

Paper 16: Key Vulnerable Nodes Discovery Using Bayesian Attack Subgraphs [16]

Description: This paper proposes identifying key vulnerable nodes using Bayesian attack subgraphs and clustering.

Limitation: The method requires structured attack graphs and high computational resources.

Solution Provided by AETHER: AETHER builds lightweight dynamic graphs from real-world reconnaissance data.

Paper 17: Automated Penetration Testing with Multiple Vulnerability Scanners [17]

Description: This study combines multiple vulnerability scanners to improve detection coverage.

Limitation: Combining scanners increases redundancy and false positives.

Solution Provided by AETHER: AETHER performs intelligent correlation and prioritization to reduce noise.

Paper 18: AI-Driven Attack Detection for Industrial Control Systems [18]

Description: This paper applies AI techniques to detect attacks and enhance cyber resilience in ICS environments.

Limitation: The focus is on detection rather than proactive threat enumeration.

Solution Provided by AETHER: AETHER complements detection systems with proactive reconnaissance-driven threat modeling.

Paper 19: AI-Driven Cybersecurity in IoT – TRIM-SEC Framework [19]

Description: This framework focuses on adaptive malware detection and lightweight encryption for IoT systems.

Limitation: The approach does not address attack surface enumeration.

Solution Provided by AETHER: AETHER extends AI-driven security by focusing on reconnaissance and threat enumeration.

Paper 20: AI-Driven Threat Hunting Using CNN-LSTM Models [20]

Description: This study proposes hybrid CNN-LSTM models for anomaly detection in enterprise networks.

Limitation: The system operates post-compromise and lacks proactive analysis.

Solution Provided by AETHER: AETHER focuses on pre-attack reconnaissance and enumeration.

Paper 21: Comparative Analysis of Automated and Manual Penetration Testing [21]

Description: This paper compares automated scanning and manual penetration testing techniques.

Limitation: The study highlights inefficiencies and alert overload in automated tools.

Solution Provided by AETHER: AETHER reduces analyst burden through AI-assisted prioritization.

Paper 22: Techno-Human Vulnerability Analysis in ICS [22]

Description: This work analyzes cybersecurity risks considering both technical and human factors.

Limitation: The approach lacks automated threat enumeration mechanisms.

Solution Provided by AETHER: AETHER automates technical threat enumeration to complement human-centric analysis.

Paper 23: Managing Cyber Harm – Survey of Challenges [23]

Description: This paper surveys cyber harm management practices and challenges.

Limitation: The study is conceptual and lacks implementation-oriented solutions.

Solution Provided by AETHER: AETHER provides an implementable platform for proactive risk identification.

Paper 24: Building Cyber-Resilience in Maritime Transport [24]

Description: This paper examines cyber resilience strategies in maritime transport systems.

Limitation: The work focuses on policy-level mitigation rather than technical enumeration.

Solution Provided by AETHER: AETHER supports technical threat enumeration adaptable to critical infrastructure.

Paper 25: AgCyRAG – Agentic Knowledge Graph-based RAG Framework [25]

Description: This framework integrates knowledge graphs and retrieval-augmented generation for security analysis.

Limitation: The approach lacks automated attack surface discovery.

Solution Provided by AETHER: AETHER integrates RAG-based reasoning with continuous reconnaissance and enumeration.

2.3 METHODS USED IN PREVIOUS STUDIES

Previous studies primarily employed standalone vulnerability scanners, static crawlers, or theoretical graph models. Data collection and analysis were often treated as separate stages, with limited automation and minimal visualization. AI techniques were either absent or applied without grounding in real reconnaissance data.

2.4 COMPARATIVE ANALYSIS OF RELATED WORK

Most existing solutions address specific cybersecurity stages such as scanning, detection, or attack path modeling. However, they lack unified integration across reconnaissance, enumeration, and intelligent prioritization.

2.5 LIMITATIONS OF EXISTING RESEARCH / SYSTEMS

Existing research suffers from fragmentation, lack of visualization, limited scalability, insufficient legal controls, and weak contextual intelligence. Most systems either overwhelm users with raw data or rely on assumptions that do not hold in real-world web environments.

2.6 SUMMARY OF FINDINGS FROM LITERATURE

The literature indicates a clear need for an integrated reconnaissance platform that unifies discovery, analysis, visualization, and reasoning. It also highlights the importance of safety, legality, and explainability, which are insufficiently addressed in existing solut**ions.**

2.7 CHAPTER SUMMARY

This chapter reviewed over twenty research works related to web reconnaissance, attack surface analysis, security automation, and AI-assisted cybersecurity. The limitations of existing systems were identified, and the proposed system, AETHER, was positioned as a comprehensive solution addressing these gaps. The next chapter presents the system architecture and methodology of AETHER.

CHAPTER 3

PROBLEM STATEMENT

3.1 STATEMENT OF THE PROBLEM

Modern organizations face increasing difficulty in maintaining a clear and accurate understanding of their external web attack surface. Rapid application development, third-party integrations, dynamic client-side content, and decentralized deployment models continuously introduce new exposed assets such as endpoints, APIs, subdomains, and undocumented pages. Existing security tools and research approaches largely operate in isolation, producing fragmented and unstructured outputs that are difficult to correlate and interpret.

Despite extensive research on web crawling (I, II), attack graph modeling (III, IV), and vulnerability scanning (V, VI), there is no unified system that integrates authorized reconnaissance, structured visualization, and contextual reasoning into a single operational platform. This gap results in poor visibility, inefficient analysis, and increased risk of overlooked security exposures.

3.2 PROBLEM DESCRIPTION

The problem addressed by this project can be described across four major dimensions:

- **Fragmented Reconnaissance Data** Existing scanners and crawlers generate large volumes of logs and reports without providing a consolidated representation of discovered assets and their relationships (V, X, XXI).
- **Lack of Structural Visibility** Most systems present findings as flat lists or textual reports, failing to represent the structural relationships between web pages, endpoints, and technologies (II, VII, XIII).

- **Limited Contextual Intelligence** Security analysts must manually interpret scan results, as traditional tools lack contextual reasoning or explainability mechanisms (XV, XVII).
- **Operational and Legal Constraints** Many reconnaissance approaches fail to enforce authorization, safety boundaries, or resource controls, increasing legal and ethical risks (XIX, XXII).

As a result, organizations are unable to efficiently map their attack surfaces or prioritize remediation efforts in a scalable and defensible manner.

3.3 RESEARCH / SYSTEM CHALLENGES

Based on the literature review, the following challenges are identified:

- **Scalability Challenge:** Focused crawling and scanning must scale without overwhelming infrastructure or violating rate limits (I, XI).
- **Data Integration Challenge:** Outputs from heterogeneous tools must be normalized into a unified data model (V, IX).
- **Visualization Challenge:** Representing complex web structures and relationships in an interpretable and interactive format remains difficult (II, XIII).
- **Automation Safety Challenge:** Automated reconnaissance must enforce authorization, scope boundaries, and execution isolation (XIX, XXII).
- **AI Reliability Challenge:** AI-based analysis systems risk hallucination and misinformation when not grounded in verified data (XV, XVI).

Addressing these challenges simultaneously is necessary to build a practical and trustworthy reconnaissance platform.

3.4 PROPOSED SOLUTION OUTLINE

To address the identified problems and challenges, this project proposes **AETHER**, a modular and legally compliant cybersecurity reconnaissance platform with the following key components:

- **Focused Crawling Engine** A controlled crawler discovers web assets and builds structured sitemaps while respecting authorization and scope constraints (I, XI).
- **Graph-Based Asset Representation** Discovered assets are represented as nodes and edges, enabling visual analysis of relationships and exposure paths (II, III).
- **Sandboxed Tool Orchestration** Security reconnaissance tools are executed in isolated worker environments with enforced timeouts and sanitized inputs (IX, XIX).
- **Unified Data Storage and Normalization** Crawl and scan outputs are converted into structured formats for consistent querying and visualization (V, XIV).
- **AI-Assisted Reasoning via RAG** A Retrieval-Augmented Generation layer allows users to query project-specific data using natural language while remaining grounded in actual evidence (XV, XVI).

Through this architecture, AETHER transforms raw reconnaissance data into actionable, explainable, and visually interpretable intelligence.

3.5 CHAPTER SUMMARY

This chapter defined the core problem addressed by the AETHER project: the lack of an integrated, scalable, and interpretable system for authorized web attack surface reconnaissance. Drawing from existing research on focused crawling (I), web graph structures (II), attack graph analysis (III, IV), vulnerability scanning (V, VI), and AI-assisted security analysis (XV, XVI), several limitations and challenges were identified. The proposed solution, AETHER, addresses these gaps by unifying crawling, tool orchestration, graph visualization, and AI-driven reasoning within a secure and legally compliant framework. The next chapter presents the detailed system architecture and methodology adopted for implementing the proposed solution.

CHAPTER 4

OBJECTIVES

4.1 PRIMARY OBJECTIVES

The primary objectives of the AETHER project are focused on addressing the core challenges identified in earlier chapters and establishing a robust foundation for authorized cybersecurity reconnaissance. The primary objectives are as follows:

1. To design and develop a unified platform for **authorized web attack surface discovery and reconnaissance**.
2. To implement a **focused web crawling mechanism** capable of discovering security-relevant web assets while minimizing unnecessary data collection.
3. To construct **graph-based representations** of web assets and their relationships for enhanced visibility and interpretability.
4. To enable **controlled orchestration of cybersecurity reconnaissance tools** within a sandboxed and resource-limited execution environment.
5. To ensure **legal compliance, safety, and authorization enforcement** throughout the reconnaissance process.
6. To integrate **AI-assisted reasoning** using Retrieval-Augmented Generation (RAG) for contextual analysis and explanation of collected security data.

4.2 SECONDARY OBJECTIVES

The secondary objectives support the primary goals by enhancing usability, scalability, and analytical depth:

1. To provide an **interactive and user-friendly frontend** for managing projects, scans, and results visualization.
2. To normalize and store reconnaissance outputs in **structured and queryable data formats**.
3. To support **historical tracking and comparison** of reconnaissance results across multiple scans.
4. To enable **natural language querying** of reconnaissance data through a conversational interface.

5. To incorporate **basic prioritization and risk indicators** based on asset relevance and connectivity.
6. To ensure system **modularity and extensibility** for future integration of additional tools and analysis modules.

4.3 EXPECTED OUTCOME

Upon successful implementation, the AETHER project is expected to achieve the following outcomes:

- A functional platform capable of performing **authorized external reconnaissance** on web applications and domains.
- Generation of **structured sitemaps and interactive graphs** that clearly represent web asset relationships.
- Automated and safe execution of reconnaissance tools with **centralized result management**.
- Improved **situational awareness** through AI-assisted summaries and contextual explanations grounded in real scan data.
- Reduction in manual effort required to interpret reconnaissance results.
- A scalable and reusable framework suitable for **defensive security assessment, education, and research purposes**.

4.4 CHAPTER SUMMARY

This chapter outlined the primary and secondary objectives of the AETHER project, emphasizing its focus on authorized reconnaissance, structured visualization, and AI-assisted analysis. The expected outcomes highlight the system's ability to transform fragmented reconnaissance outputs into actionable and interpretable security intelligence. The next chapter will describe the system architecture and methodological approach adopted to achieve these objectives.

CHAPTER 5

PROPOSED SYSTEM

5.1 OVERVIEW OF PROPOSED SYSTEM

The proposed system, **AETHER**, is a modular and legally compliant cybersecurity reconnaissance platform designed to identify, map, and analyze the external attack surface of web-based applications. The system integrates focused web crawling, controlled execution of security reconnaissance tools, graph-based visualization, and AI-assisted reasoning into a unified framework.

Unlike traditional standalone scanners, AETHER emphasizes **structural visibility**, **safe automation**, and **contextual intelligence**. The system separates orchestration from execution by utilizing a serverless API layer for coordination and a sandboxed worker environment for resource-intensive reconnaissance tasks. All collected data is securely stored and later analyzed using graph models and Retrieval-Augmented Generation (RAG) techniques.

5.2 BLOCK DIAGRAM

The block diagram of the proposed system consists of the following major components:

1. User Interface (Frontend)
2. API & Orchestration Layer
3. Authentication & Database Layer
4. Worker / Execution Engine
5. Crawler & Sitemap Generator
6. Graph Visualization Engine
7. AI-Assisted Analysis (RAG Layer)

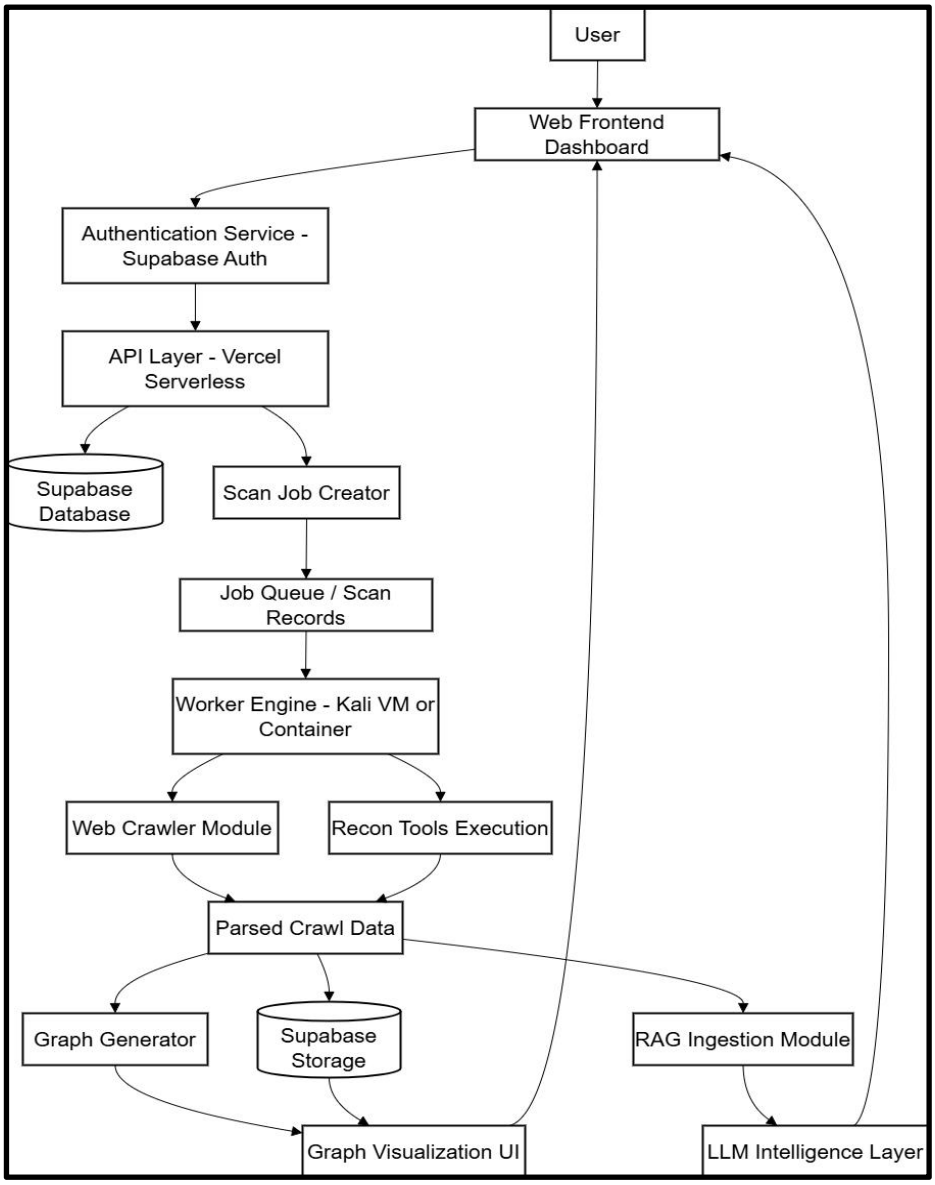

Figure 5.1: Flow Map / Block Diagram

Data Flow Description:

User → Frontend → API Layer → Job Queue → Worker Engine → Crawler / Recon Tools → Structured Data Storage → Graph Visualization + AI Reasoning → User

This flow ensures that user interactions remain lightweight while heavy reconnaissance tasks are executed in isolated environments.

5.3 SYSTEM ARCHITECTURE

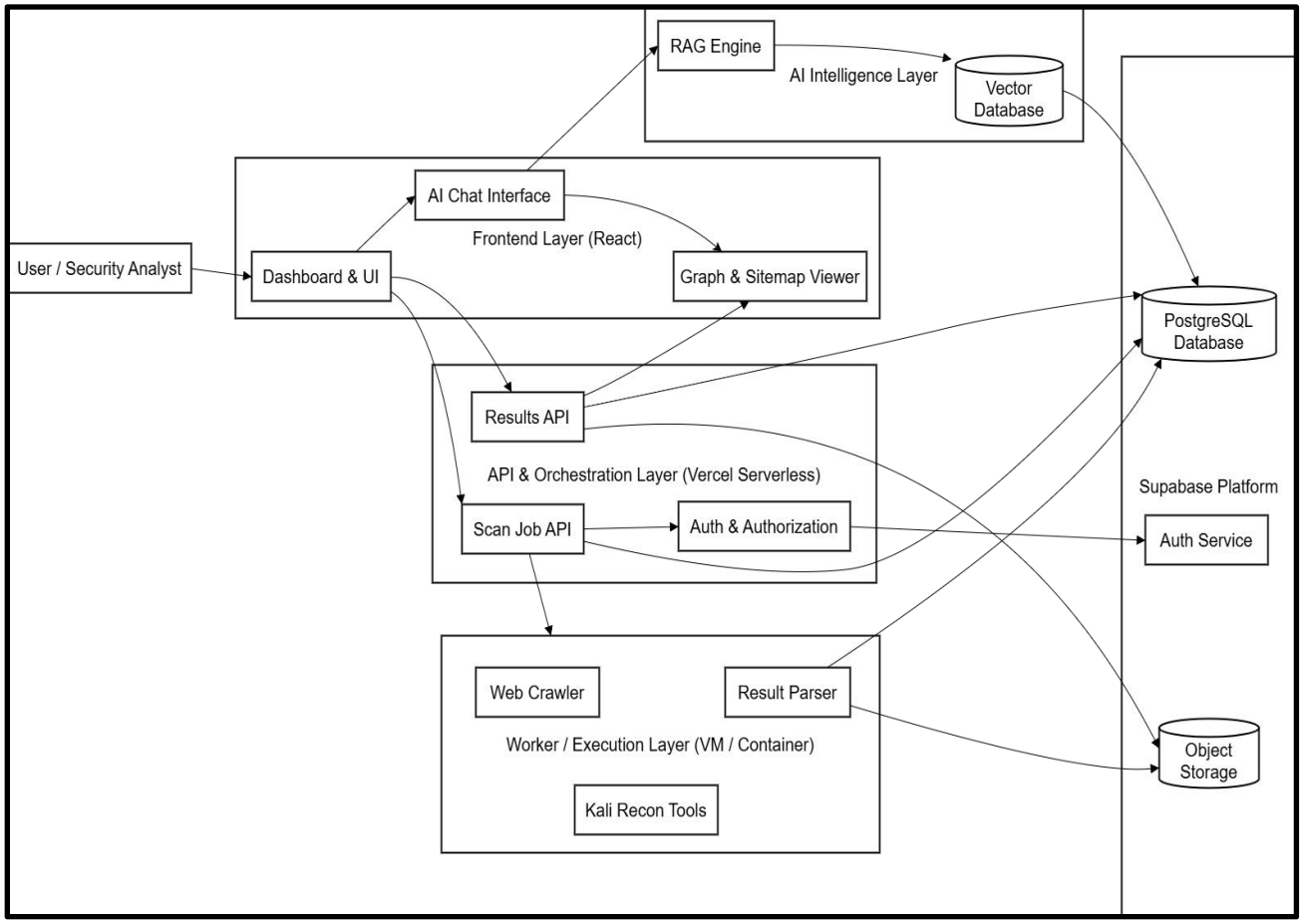

Figure 5.2: System Architecture

The system architecture of AETHER follows a **distributed, service-oriented design**, consisting of the following layers:

1. Presentation Layer

- React-based frontend
- Provides project creation, scan initiation, result visualization, and chatbot interface

2. API & Orchestration Layer

- Serverless API (Vercel)
- Handles authentication, job creation, job status queries, and result retrieval

- Does not perform long-running reconnaissance tasks

3. Authentication & Storage Layer

- Supabase Authentication for secure user access
- PostgreSQL database for metadata
- Object storage for scan outputs and crawl artifacts

4. Worker / Execution Layer

- Self-hosted VM or containerized environment
- Executes Kali Linux reconnaissance tools
- Enforces timeouts, resource limits, and input sanitization

5. Intelligence Layer

- Graph engine for asset relationship modeling
- RAG-based AI layer for contextual querying and summarization

This layered architecture ensures scalability, safety, and maintainability.

5.4 FUNCTIONAL DESCRIPTION

The functional operation of the proposed system is described below:

1. The user logs in and creates a project using the frontend interface.
2. The user submits a scan request with authorized target details.
3. The API layer validates the request and creates a scan job.
4. The worker engine retrieves the job and performs:
   a) Focused crawling
   b) Tool-based reconnaissance
5. Results are parsed and stored in structured formats.
6. A sitemap and graph representation are generated.
7. The user views interactive visualizations and queries the data using the AI interface.

Each function is executed within defined scope and safety constraints.

5.5 MODULES / COMPONENTS OVERVIEW

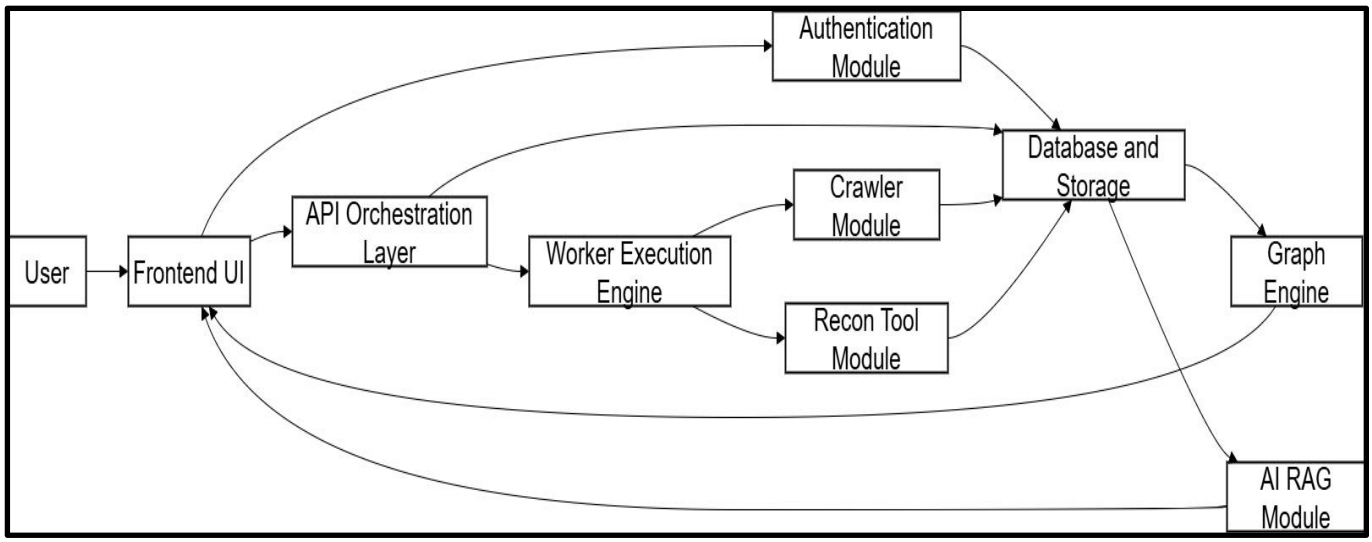

Figure 5.3: Modular Architecture View

1. User Interface Module

- Project dashboard
- Scan controls
- Graph and sitemap viewers
- Chatbot interface

2. Authentication Module

- User login and access control
- Project ownership enforcement

3. Crawler Module

- Discovers internal pages and endpoints
- Respects scope, depth, and authorization limits

4. Reconnaissance Module

- Executes tools such as Nmap, Nikto, and WhatWeb
- Collects service and configuration information

5. Data Normalization Module

- Converts raw outputs into structured JSON
- Ensures consistency across tools

6. Graph Engine Module

- Constructs node-edge models
- Supports interactive visualization

7. AI Reasoning Module

- Uses RAG to answer project-specific queries
- Grounds responses in stored reconnaissance data

5.6 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages over existing approaches:

- **Unified Platform:** Integrates crawling, scanning, visualization, and AI reasoning
- **Legal and Ethical Compliance:** Enforces authorization and scope control
- **Improved Visibility:** Graph-based visualization enhances understanding
- **Scalability:** Separation of orchestration and execution enables efficient scaling
- **Explainability:** AI responses are grounded in real data
- **Modularity:** Supports future extensions and additional tools

5.7 CHAPTER SUMMARY

This chapter presented the proposed system architecture and design of AETHER. It described the overall system flow, block diagram, architectural layers, functional operation, and core modules. The advantages of the proposed system highlight its effectiveness in providing a secure, scalable, and interpretable solution for authorized web reconnaissance. The next chapter will focus on the implementation details of the system.

CHAPTER 6

SYSTEM DESIGN

6.1 INTRODUCTION TO SYSTEM DESIGN

System design defines the structural blueprint of the AETHER platform, detailing how system components interact, how data flows across modules, and how functional and non-functional requirements are achieved. This chapter presents the Unified Modeling Language (UML) diagrams, database design, and technical requirements that collectively describe the internal working of the proposed system.

The design of AETHER follows a **modular, scalable, and secure architecture**, ensuring separation of concerns between orchestration, execution, visualization, and intelligence layers.

6.2 UML Diagrams

6.2.1 USE CASE DIAGRAM

Actors:

- User (Security Analyst / Researcher)
- System Administrator
- Worker Node (Execution Engine)

Use Cases:

- Register / Login
- Create Project
- Define Scope and Authorization
- Initiate Scan
- View Scan Results
- Visualize Attack Surface Graph
- Query Results using AI
- Manage Users (Admin)
- Monitor Worker Status

Description: The user interacts with the system to create projects, initiate scans, and analyze results. The worker node executes reconnaissance tasks, while the administrator manages system configurations.

6.2.2 CLASS DIAGRAM

Actors:

- User (Security Analyst / Researcher)
- System Administrator
- Worker Node (Execution Engine)

Use Cases:

- Register / Login
- Create Project
- Define Scope and Authorization
- Initiate Scan
- View Scan Results
- Visualize Attack Surface Graph
- Query Results using AI
- Manage Users (Admin)
- Monitor Worker Status

Description: The user interacts with the system to create projects, initiate scans, and analyze results. The worker node executes reconnaissance tasks, while the administrator manages system configurations.

6.2.3 SEQUENCE DIAGRAM

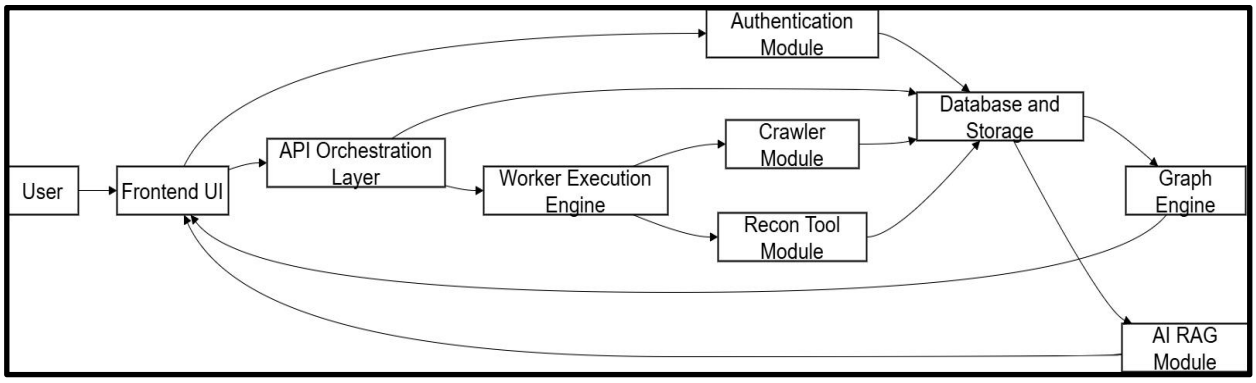

Figure 6.1: Sequence Diagram

Sequence: Initiate and Execute Scan

1. User submits scan request via Frontend
2. API validates authorization and scope
3. API creates scan job
4. Worker fetches job from queue
5. Worker executes crawler and tools
6. Results are parsed and stored
7. Graph is generated
8. User retrieves and views results

This sequence ensures asynchronous execution and responsiveness.

6.2.4 ACTIVITY DIAGRAM

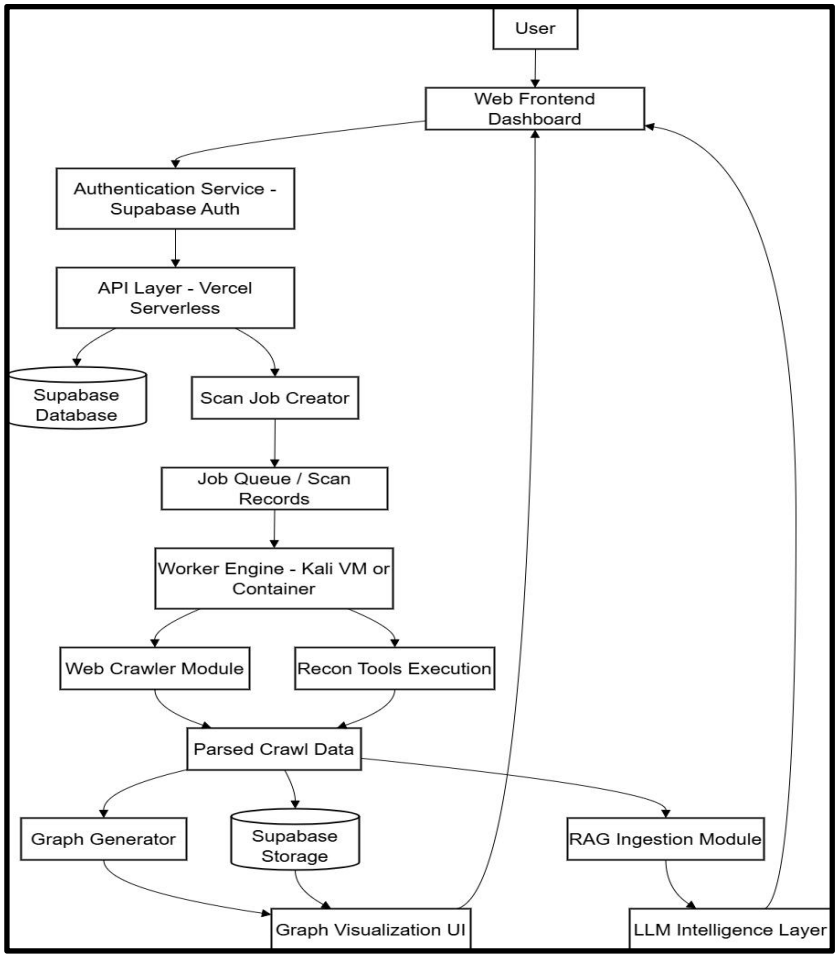

Figure 6.2: Activity Diagram

Activities:

- Start
- User Login
- Project Selection
- Scan Configuration
- Authorization Validation
- Recon Execution
- Data Processing
- Visualization
- End

Decision nodes ensure scope validation and error handling during execution.

6.2.5 COMPONENT DIAGRAM

Components:

- Frontend (React + Typescript)
- API Gateway
- Auth Service
- Job Queue
- Worker Engine
- Recon Tools
- Database
- Graph Engine
- AI Module

Each component communicates through defined interfaces, supporting loose coupling.

6.3 DATABASE DESIGN

The database design ensures efficient storage, retrieval, and correlation of reconnaissance data.

6.3.1 ER DIAGRAM

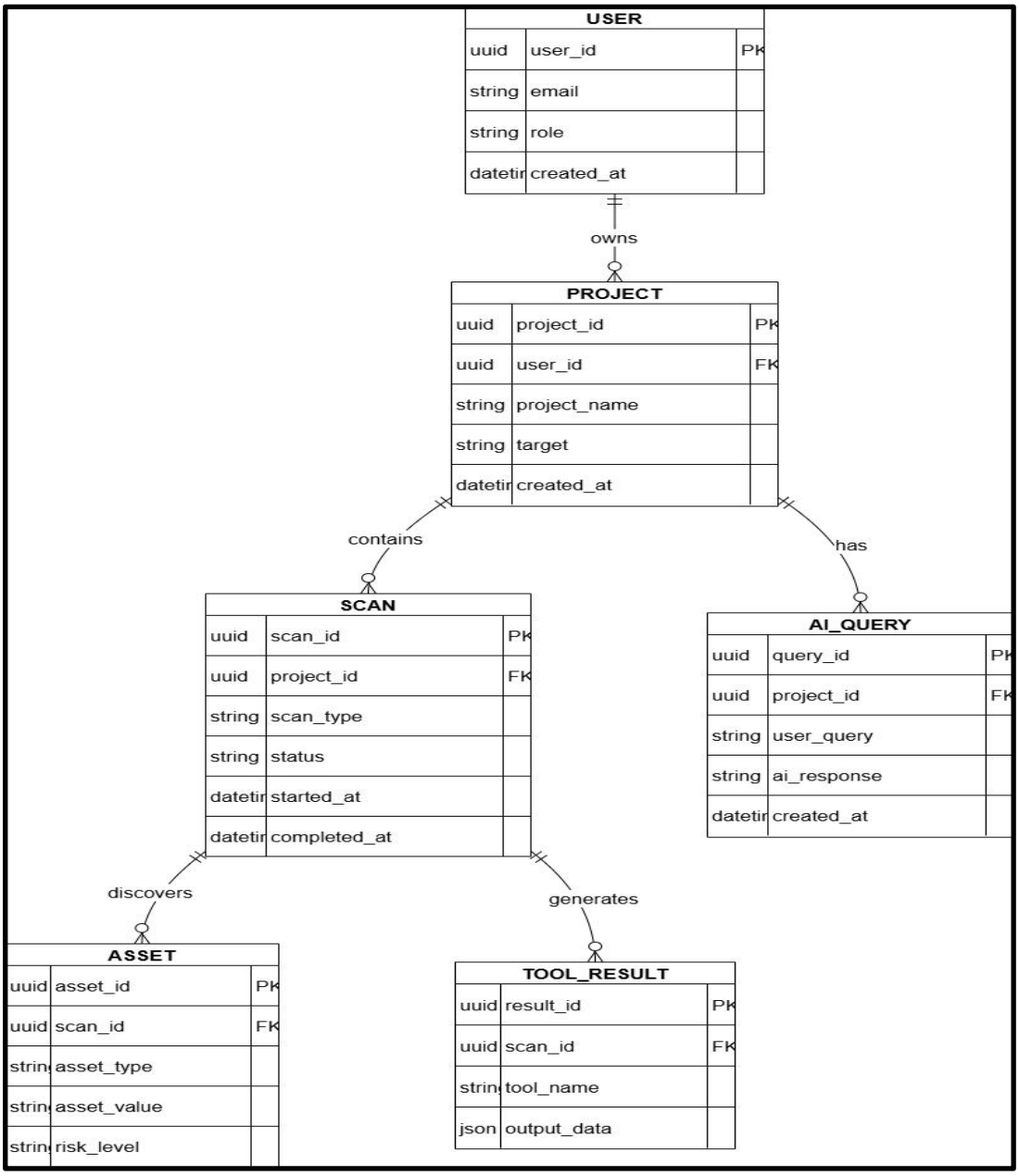

Figure 6.3: ER Diagram

Entities:

- User
- Project
- Scan
- Asset
- ToolResult
- Graph
- AIQuery

Relationships:

- User → Project (1:N)
- Project → Scan (1:N)
- Scan → Asset (1:N)
- Scan → ToolResult (1:N)

6.3.2 SCHEMA DIAGRAM

Schema Overview:

- users(user_id, email, role)
- projects(project_id, user_id, target)
- scans(scan_id, project_id, status)
- assets(asset_id, scan_id, type)
- tool_results(result_id, scan_id, tool_name)
- graph_nodes(node_id, asset_id)
- graph_edges(edge_id, source, target)
- ai_queries(query_id, scan_id)

6.3.3 TABLE DEFINITIONS

Users Table

- user_id (PK)
- email
- password_hash
- role
- created_at

Projects Table

- project_id (PK)
- user_id (FK)
- target_domain
- created_at

Scans Table

- scan_id (PK)
- project_id (FK)
- status
- started_at
- completed_at

Assets Table

- asset_id (PK)
- scan_id (FK)
- asset_type
- asset_value

Tool_Results Table

- result_id (PK)
- scan_id (FK)
- tool_name
- output_json

6.4 HARDWARE REQUIREMENTS

- Processor: Intel i5 / Ryzen 5 or higher
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 50 GB free disk space
- Network: Stable internet connection
- Optional: Cloud VM for worker execution

6.5 SOFTWARE REQUIREMENTS

- Operating System: Linux (Ubuntu/Kali), Windows for frontend
- Frontend: React.js, Tailwind CSS
- Backend: Node.js / Serverless Functions
- Database: PostgreSQL (Supabase)
- Containerization: Docker
- Recon Tools: Nmap, Nikto, WhatWeb
- AI: LLM with RAG integration
- Visualization: D3.js / Cytoscape.js

6.6 CHAPTER SUMMARY

This chapter presented the detailed system design of the AETHER platform, including UML diagrams, database architecture, and technical requirements. The design emphasizes modularity, scalability, and security, ensuring that AETHER can efficiently perform authorized reconnaissance while maintaining data integrity and user safety. The next chapter will discuss the implementation details and execution workflow of the system.

CHAPTER 7

MODULE DESCRIPTION

The AETHER system is designed using a modular architecture to ensure scalability, maintainability, and clarity of responsibilities. Each module performs a specific function within the overall system while interacting seamlessly with other modules. This chapter describes the objectives, functionality, inputs, outputs, algorithms, interfaces, and APIs used in each module.

7.1 User Management Module

7.1.1 Module Objectives

- Provide secure authentication and authorization.
- Manage user roles and access privileges.
- Maintain session integrity.
- Protect system endpoints from unauthorized access.

7.1.2 Module Overview

The User Management Module controls access to the AETHER platform. It handles account creation, authentication, token validation, and session handling. Security mechanisms ensure only authenticated users initiate reconnaissance operations.

7.1.3 Input & Output Description

Input:

- Username / Email
- Password
- Role Information
- Login Requests

Output:

- JWT Authentication Token
- User Session Status
- Authorization Response

7.1.4 Algorithms / Logic Used

- Password Hashing (bcrypt hashing)
- JWT Token Generation
- Role-Based Access Control (RBAC)
- Session Validation Logic
- Token Expiry Verification

7.1.5 API / Functions Used

- POST /api/auth/register
- POST /api/auth/login
- GET /api/auth/profile
- POST /api/auth/logout

Functions:

- create_user()
- authenticate_user()
- generate_token()
- verify_token()

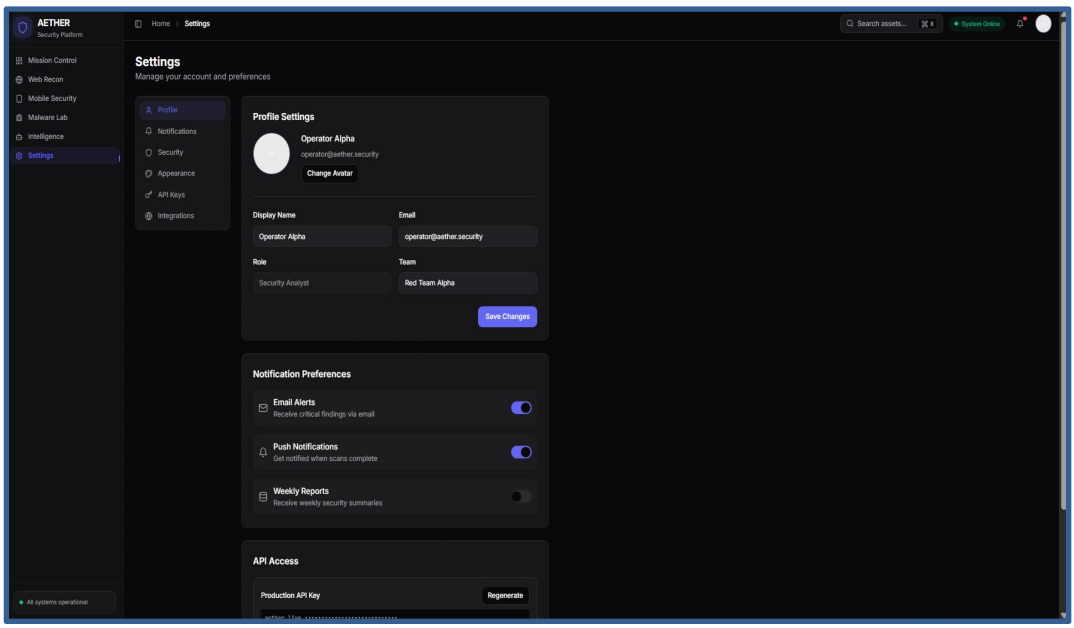

Figure 7.1: User Settigns

7.2 Project Management Module

7.2.1 Module Objectives

- Manage reconnaissance projects.
- Define authorized testing scope.
- Store target assets securely.
- Track project lifecycle.

7.2.2 Module Overview

This module allows users to create and manage projects representing authorized targets. Each project includes domains, IP ranges, and scope definitions used by scanning modules.

7.2.3 Input & Output Description

Input:

- Target Domain
- Scope Definition
- Authorization Documents

Output:

- Project ID
- Scope Validation Status
- Project Metadata

7.2.4 Algorithms / Logics Used

- Scope Validation Logic
- Target Deduplication Algorithm
- Project State Management
- Authorization Verification

7.2.5 API / Functions Used

- POST /api/projects
- GET /api/projects
- GET /api/projects/{id}
- DELETE /api/projects/{id}

Functions:

- create_project()
- validate_scope()
- update_project()

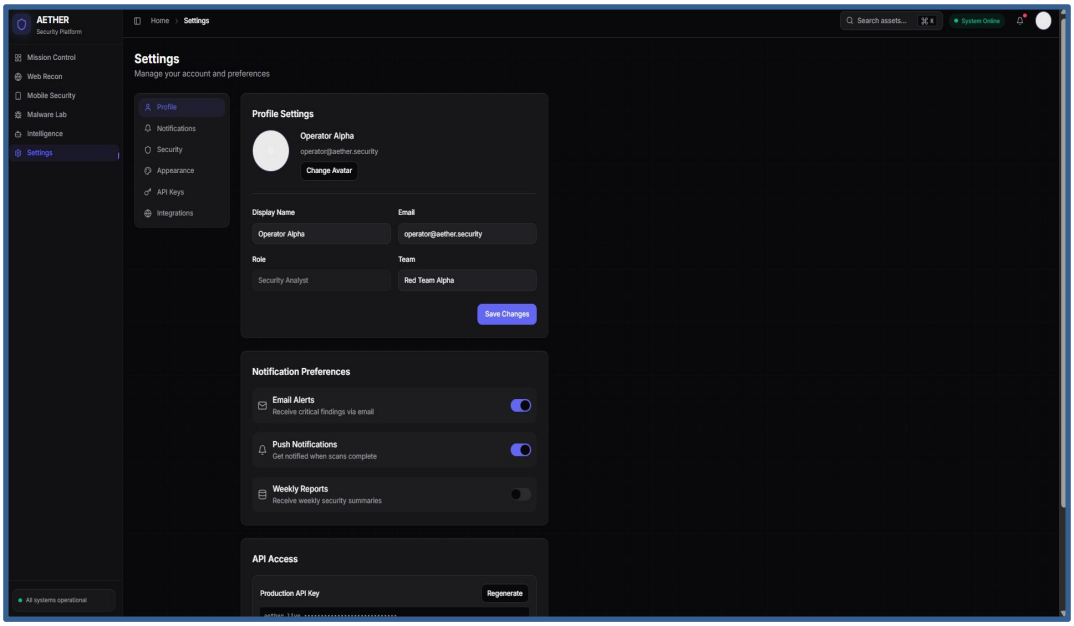

Figure 7.2: Project Management

7.3 Reconnaissance Automation Module

7.3.1 Module Objectives

- Discover external attack surface.
- Automate reconnaissance tasks.
- Collect intelligence efficiently.

7.3.2 Module Overview

The Reconnaissance Automation Module performs automated enumeration of assets related to a target domain. It integrates multiple reconnaissance tools executed through worker nodes.

7.3.3 Input & Output Description

Input:

- Target Domain
- Scan Configuration
- Project ID

Output:

- Subdomains
- DNS Records
- Identified Technologies
- Asset Intelligence Dataset

7.3.4 Algorithms / Logics Used

- Passive Recon Logic
- Active Enumeration Workflow
- Multi-tool Result Aggregation
- Parallel Task Execution

7.3.5 API / Functions Used

- POST /api/recon/start
- GET /api/recon/status/{id}
- GET /api/recon/results/{id}

Functions:

- start_scan()
- enumerate_subdomains()
- collect_dns_data()
- parse_results()

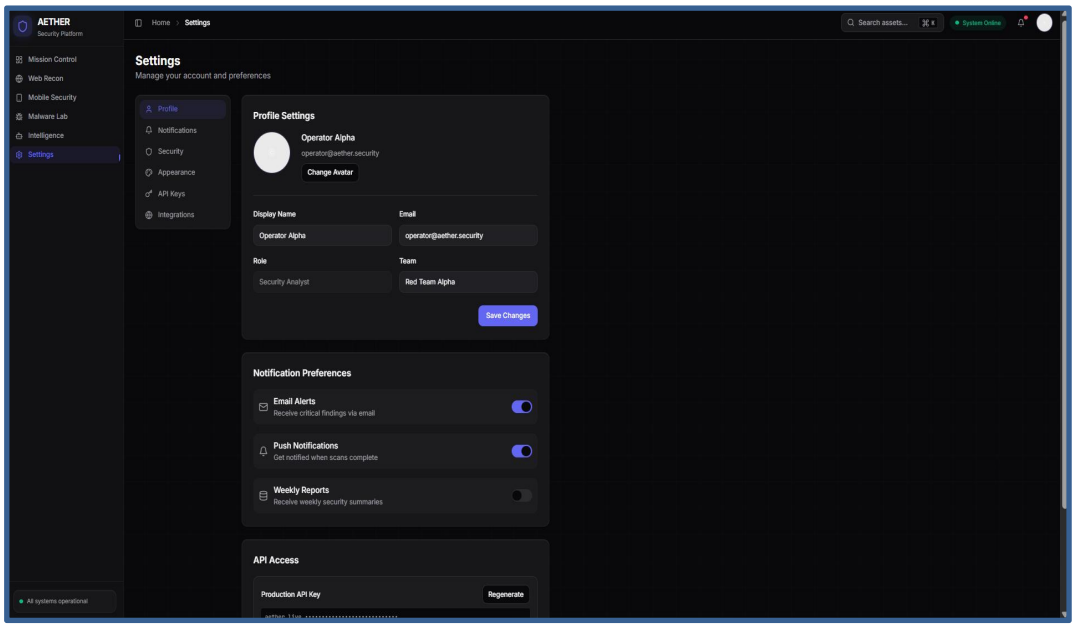

Figure 7.3: Recon

7.4 Web Crawling Module

7.4.1 Module Objectives

- Identify hidden endpoints.
- Discover application routes.
- Expand attack surface mapping.

7.4.2 Module Overview

This module crawls web applications to collect URLs, parameters, and API routes. It simulates browser behavior to discover dynamically loaded resources.

7.4.3 Input & Output Description

Input:

- Target URL
- Crawl Depth
- Scan ID

Output:

- Discovered URLs
- API Endpoints
- Parameter Lists

7.4.4 Algorithms / Logics Used

- Breadth-First Crawling
- Recursive Link Traversal
- JavaScript Endpoint Extraction
- URL Normalization Logic

7.4.5 API / Functions Used

- POST /api/crawler/start
- GET /api/crawler/results/{id}

Functions:

- crawl_site()
- extract_links()
- analyze_scripts()

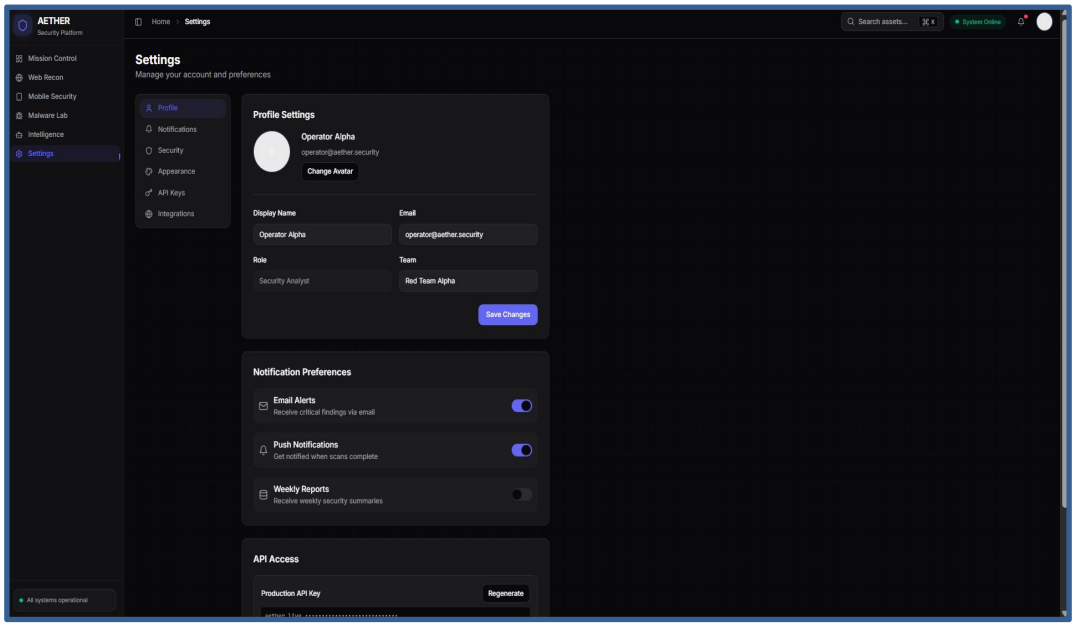

Figure 7.4: Web Crawl

7.5 Mobile Security Analysis Module

7.5.1 Module Objectives

- Analyze Android applications securely.
- Detect exposed components.
- Identify insecure configurations

7.5.2 Module Overview

This module performs static analysis of APK files using sandboxed environments without executing the application.

7.5.3 Input & Output Description

Input:

- APK File
- Analysis Configuration

Output:

- Permission Report
- Manifest Analysis
- Vulnerability Indicators

7.5.4 Algorithms / Logics Used

- Static Code Analysis
- Manifest Parsing Logic
- Permission Risk Scoring
- Component Enumeration

7.5.5 API / Functions Used

- POST /api/mobile/upload
- GET /api/mobile/report/{id}

Functions:

- analyze_apk()
- extract_manifest()
- evaluate_permissions()

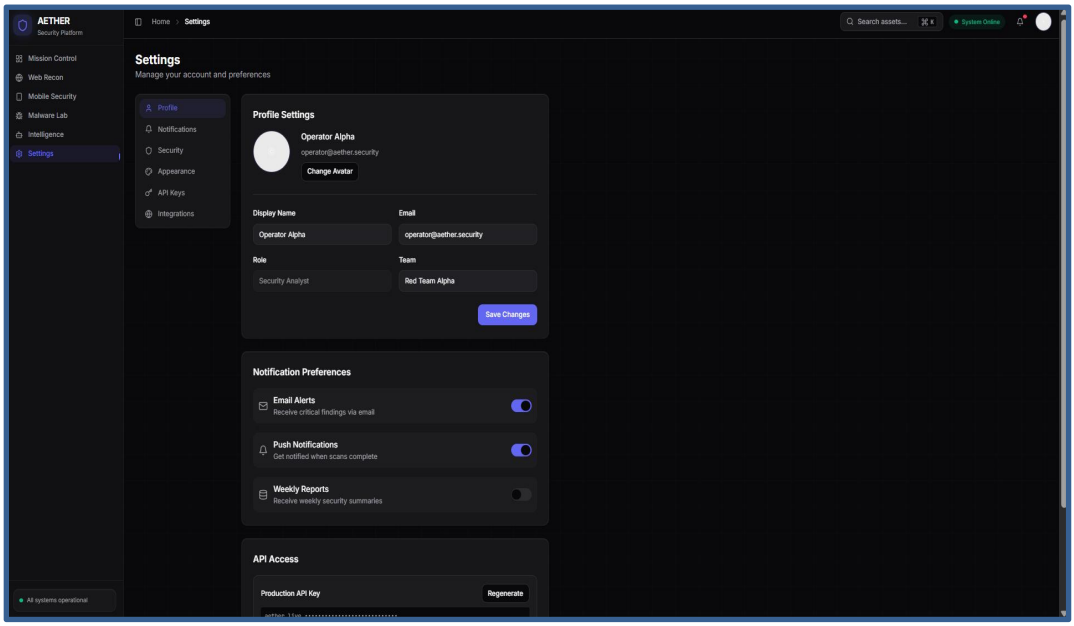

Figure 7.5: Mobile Security

7.6 Graph Visualization Module

7.6.1 Module Objectives

- Visualize attack surface relationships.
- Improve analyst understanding.
- Enable interactive exploration.

7.6.2 Module Overview

This module converts reconnaissance data into graph structures representing relationships between domains, endpoints, technologies, and vulnerabilities.

7.6.3 Input & Output Description

Input:

- Reconnaissance Dataset
- Asset Relationships

Output:

- Interactive Graph Visualization
- Node Relationship Maps

7.6.4 Algorithms / Logics Used

- Graph Construction Algorithm
- Node-Edge Mapping
- Network Visualization Logic
- Data Aggregation Pipeline

7.6.5 API / Functions Used

- GET /api/graph/{project_id}

Functions:

- generate_graph()
- map_relationships()
- render_nodes()

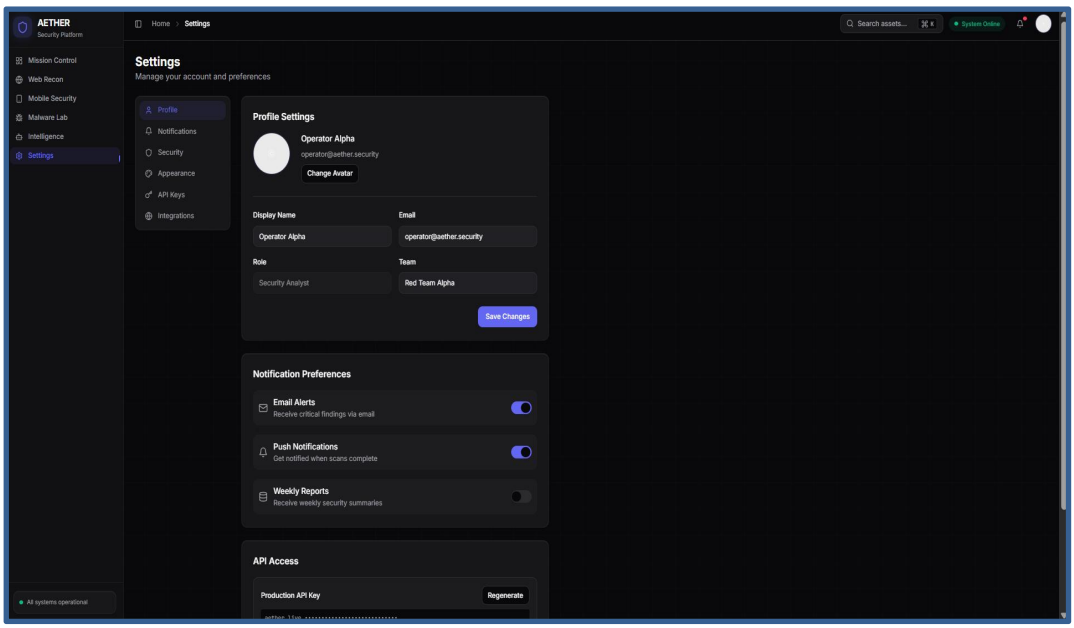

Figure 7.6: Graph Visualization

7.7 AI Intelligence Module

7.7.1 Module Objectives

- Provide AI-assisted analysis.
- Enable natural language interaction.
- Generate contextual security insights.

7.7.2 Module Overview

The AI Intelligence Module uses Retrieval-Augmented Generation (RAG) to interpret reconnaissance data and answer analyst queries.

7.7.3 Input & Output Description

Input:

- User Query
- Recon Data Embeddings

Output:

- AI Insights
- Security Recommendations
- Summarized Reports

7.7.4 Algorithms / Logics Used

- Vector Embedding Generation
- Semantic Similarity Search
- RAG Pipeline Logic
- Contextual Response Generation

7.7.5 API / Functions Used

- POST /api/ai/query
- POST /api/ai/summarize

Functions:

- embed_documents()
- retrieve_context()
- generate_response()

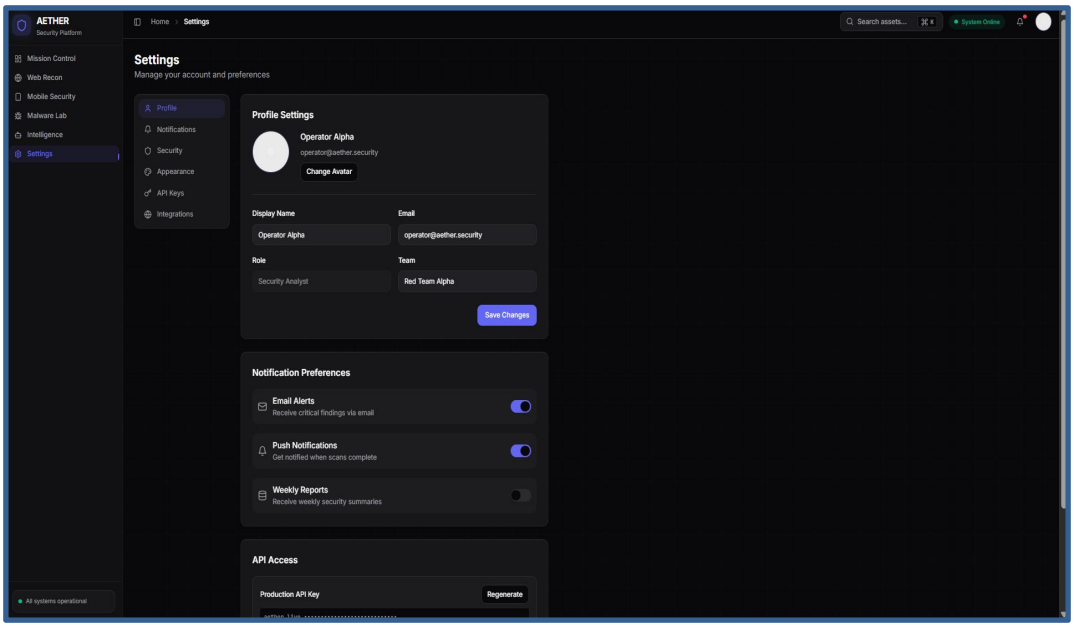

Figure 7.7: AI Intelligence

7.8 Job Queue & Worker Module

7.8.1 Module Objectives

- Execute long-running tasks asynchronously.
- Improve system scalability.
- Manage distributed workloads.

7.8.2 Module Overview

Background workers process scanning jobs independently using task queues, preventing UI blocking and improving performance.

7.8.3 Input & Output Description

Input:

- Scan Job Requests
- Task Parameters

Output:

- Job Status
- Execution Logs
- Completed Results

7.8.4 Algorithms / Logics Used

- Asynchronous Task Scheduling
- Queue Management Logic
- Retry & Failure Handling
- Worker Load Distribution

7.8.5 API / Functions Used

- POST /api/jobs/create
- GET /api/jobs/status/{id}

Functions:

- enqueue_job()
- execute_worker()
- update_status()

7.9 Data Storage Module

7.9.1 Module Objectives

- Store reconnaissance data efficiently.
- Maintain historical scan results.
- Support AI embeddings storage.

7.9.2 Module Overview

The system uses structured databases, object storage, and vector databases to manage different data types.

7.9.3 Input & Output Description

Input

- Scan Results
- Metadata
- AI Embeddings

Output

- Persisted Records
- Queryable Datasets

7.9.4 Algorithms / Logics Used

- ORM Mapping Logic
- Data Indexing
- Vector Similarity Search
- Backup & Recovery Logic

7.9.5 API / Functions Used

- GET /api/data/results/{project_id}

Functions:

- store_results()
- retrieve_results()
- save_embeddings()

7.10 CHAPTER SUMMARY

This chapter detailed the functional modules of the AETHER system, outlining their objectives, workflows, inputs, outputs, algorithms, interfaces, and APIs. The modular design ensures scalability, security, and ease of enhancement. Each module collectively contributes to building a comprehensive, AI-assisted cybersecurity reconnaissance platform.

CHAPTER 8

IMPLEMENTATION

8.1 Implementation Details

The implementation of **AETHER** follows a modular and layered approach to ensure scalability, maintainability, and secure execution of reconnaissance tasks. The system is implemented using a combination of frontend technologies for user interaction, serverless APIs for orchestration, and isolated worker environments for executing reconnaissance tools.

The frontend is responsible for user authentication, project management, scan initiation, visualization of results, and interaction with the AI-based query system. The backend API layer handles request validation, authorization checks, job creation, and coordination between the frontend and the worker nodes. The worker module is implemented as a separate execution environment to safely run security tools and crawlers without exposing the core system to risks associated with long-running or resource-intensive processes.

All collected data is normalized and stored in a centralized database and object storage, enabling efficient retrieval, visualization, and AI-assisted analysis.

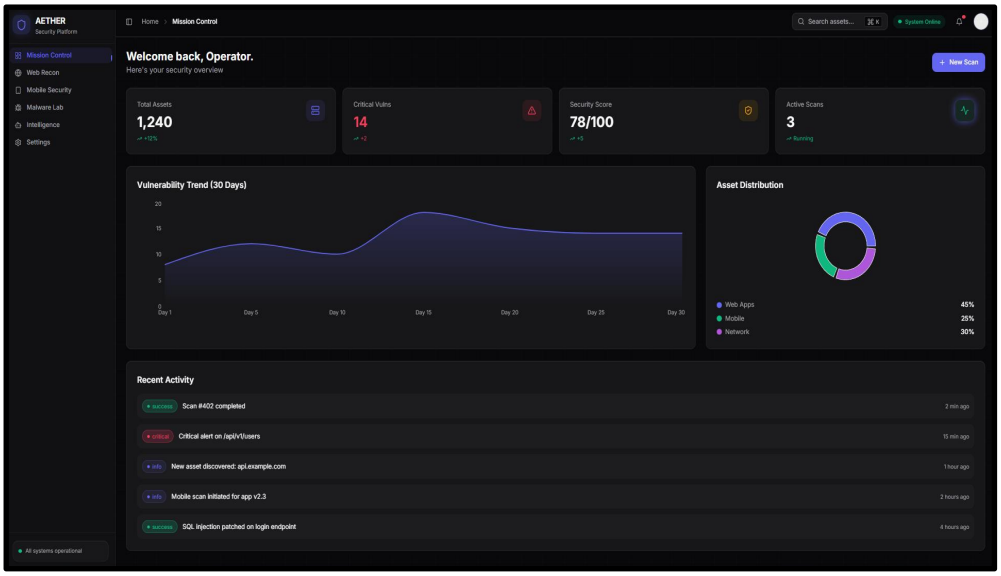

Figure 8.1: Dashboard

8.2 Technology Stack

The system was developed using industry-standard development tools and frameworks to ensure portability and performance.

Hardware Requirements

- Processor : Intel i5 / Ryzen 5 or above
- RAM : Minimum 8 GB
- Storage : 256 GB SSD
- Network : Stable Internet Connection

Software Requirements

- **Operating System :** Linux / Windows
- **Programming Language :** Python
- **Backend Framework :** FastAPI
- **Frontend Framework :** React.js
- **Database :** PostgreSQL
- **Queue System :** Redis
- **Containerization :** Docker
- **Version Control :** Git

8.3 System Architecture Implementation

The AETHER platform follows a layered architecture consisting of:

1. Presentation Layer
2. API Layer
3. Processing Layer
4. Worker Layer
5. Data Layer
6. AI Intelligence Layer

Implementation Flow

1. User accesses dashboard.
2. Authentication service validates identity.
3. Project scope is verified.
4. Recon job created.
5. Worker services execute scans asynchronously.
6. Results stored in database.
7. Graph and AI modules process outputs.
8. Dashboard visualizes findings.

8.4 Backend Implementation

The backend was implemented using FastAPI to provide high-performance asynchronous APIs.

Features Implemented

- REST API development
- Authentication middleware
- Background job processing
- Data validation
- Secure request handling

Backend Components

- Authentication Service
- Project Controller
- Recon Controller
- AI Service Controller
- Job Queue Manager

Implementation Highlights

- Async endpoints reduce blocking operations.
- Dependency injection ensures modular coding.
- Centralized exception handling improves reliability.

8.5 Frontend Implementation

The frontend interface provides an interactive environment for analysts.

Implemented Features

- Dashboard Interface
- Project Management Panel
- Scan Monitoring Interface
- Graph Visualization Panel
- AI Chat Assistant

Technologies Used

- React.js
- Tailwind CSS
- Axios API Integration
- Dynamic State Management

UI Workflow

- User logs in.
- Creates project.
- Starts reconnaissance.
- Views real-time results.
- Queries AI assistant.

8.6 Database Implementation

The system uses PostgreSQL for structured storage.

Database Tables Implemented

- Users
- Projects
- Recon Results
- Crawled Endpoints
- Scan Jobs
- AI Embeddings Metadata

Database Features

- Indexed queries for performance.
- Foreign key constraints.
- Transaction handling.
- Secure credential storage.
-

8.7 Job Queue Implementation

Long-running tasks are handled asynchronously.

Queue Architecture

- Redis Message Broker
- Worker Nodes
- Task Scheduler

Execution Steps

- Scan request added to queue.
- Worker retrieves task.
- Recon tools executed.
- Results stored.
- Status updated.

Benefits

- Non-blocking execution
- Parallel processing
- Fault tolerance

8.8 Reconnaissance Tool Integration

External reconnaissance utilities were integrated into the system.

Integrated Functionalities

- Subdomain Enumeration
- DNS Resolution
- Technology Detection

Endpoint Discovery

- **Integration Method**
- Python subprocess execution
- Output parsing
- Standardized JSON result conversion

8.9 AI Intelligence Implementation

The AI subsystem provides contextual cybersecurity analysis.

Implementation Components

- Embedding Generator
- Vector Database Storage
- Retrieval-Augmented Generation (RAG)
- Natural Language Query Interface

Processing Workflow

- Recon results converted into embeddings.
- Stored in vector database.
- Query retrieves relevant context.
- AI generates analytical response.

8.10 Graph Visualization Implementation

Graph visualization enhances analyst understanding.

Implementation Features

- Node-edge mapping
- Relationship clustering
- Interactive filtering
- Dynamic rendering

Data Processing Steps

- Normalize assets.
- Map relationships.
- Generate graph schema.
- Render visualization.

8.11 Security Implementation

Security mechanisms implemented include:

- JWT Authentication
- Password Hashing
- Role-Based Access Control
- API Rate Limiting
- Secure Scan Isolation
- Authorization Validation

All reconnaissance activities require validated project scope before execution.

8.12 API Implementation

The system exposes RESTful APIs enabling modular communication.

Core API Categories

- Authentication APIs
- Project APIs
- Recon APIs
- AI APIs
- Visualization APIs
- Job APIs

All APIs follow JSON-based request/response models.

8.13 Deployment Implementation

The application is containerized for consistent deployment.

Deployment Stack

- Docker Containers
- Reverse Proxy Configuration
- Environment Variable Management
- Service Isolation

Deployment Steps

- Build Docker Images.
- Configure services.
- Initialize databases.
- Start workers.
- Launch frontend server.

8.14 Testing Implementation

Testing ensured reliability and correctness.

Testing Types

- Unit Testing
- API Testing
- Integration Testing
- Functional Testing
- Security Testing

Automated test scripts validate API responses and module interactions.

8.15 Implementation Challenges

Key challenges encountered:

- Handling long-running scans.
- Managing asynchronous workflows.
- Standardizing multi-tool outputs.
- Maintaining secure execution environments.
- Solutions involved queue-based architecture and modular integration.

8.16 Chapter Summary

This chapter detailed the implementation of the AETHER system, including the technology stack, system configuration, execution flow, and testing procedures. The modular architecture and secure execution model ensure efficient reconnaissance while maintaining legal and ethical compliance. The successful implementation validates the feasibility of integrating automated reconnaissance, graph-based visualization, and AI-assisted analysis into a unified cybersecurity intelligence platform. The next chapter discusses the results, performance evaluation, and comparative analysis of the system.

CHAPTER 9

RESULTS AND DISCUSSIONS

9.1 Introduction

This chapter presents the operational results obtained after implementing the AETHER cybersecurity reconnaissance platform. The results demonstrate how automated reconnaissance, AI-assisted analysis, and visualization improve attack surface understanding and security assessment efficiency.

The discussion evaluates system performance, usability, automation capability, and analytical effectiveness.

9.2 System Execution Results

The system was executed under controlled testing environments using authorized targets.

Observed Outcomes

- Successful user authentication and session management.
- Projects created with validated scope authorization.
- Automated reconnaissance completed without manual intervention.
- Real-time monitoring of scan progress.
- AI module generated contextual insights.
- Graph visualization displayed relationships between discovered assets.

The results confirm the successful integration of all modules into a unified platform.

9.3 Reconnaissance Results

The reconnaissance automation module identified multiple digital assets associated with target domains.

Typical Discoveries

- Subdomains and host assets.
- Web endpoints and APIs.
- Technology stack identification.
- DNS intelligence information.
- Hidden application routes.

Automation significantly reduced manual enumeration time compared to traditional methods.

9.4 Web Crawling Analysis Results

The crawling module successfully explored application structures.

Key Findings

- Discovery of dynamic endpoints.
- Identification of parameterized URLs.
- Mapping of application navigation paths.
- Detection of unused or hidden resources.

Recursive crawling expanded the observable attack surface beyond manually accessible pages.

9.5 Mobile Application Analysis Results

The mobile analysis module evaluated Android applications through static inspection.

Results Obtained

- Permission risk identification.
- Exported component detection.
- Manifest configuration evaluation.
- Security misconfiguration discovery.

The analysis provided early indicators of potential vulnerabilities without executing application code.

9.6 AI Intelligence Results

The AI Intelligence module enhanced analyst productivity.

Observed Capabilities

- Natural language querying of scan results.
- Context-aware explanations.
- Automated summarization of findings.
- Risk interpretation assistance.

Users were able to understand reconnaissance outcomes faster through AI-assisted insights.

9.7 Visualization Results

The graph visualization module produced interactive attack surface maps.

Visualization Benefits

- Clear relationship mapping between assets.
- Identification of infrastructure dependencies.
- Improved situational awareness.
- Faster vulnerability investigation workflow.

Graph-based representation simplified complex reconnaissance datasets.

9.8 Performance Evaluation

System performance was evaluated based on responsiveness and workload handling.

Performance Observations

- Fast API response times.
- Stable background job execution.
- Efficient database querying.
- Scalable task execution using worker nodes.

The asynchronous architecture ensured uninterrupted user interaction during long-running scans.

9.9 Usability Analysis

User interaction testing showed positive usability outcomes.

Usability Findings

- Simple navigation workflow.
- Clear dashboard organization.
- Minimal learning curve.
- Effective visualization and AI assistance.

The interface supports both beginner analysts and experienced security researchers.

9.10 Comparative Discussion

Compared to traditional reconnaissance workflows:

Parameter	Traditional Method	AETHER System
Automation	Low	High
Analysis Speed	Slow	Fast
Visualization	Manual	Automated
AI Assistance	None	Integrated
Scalability	Limited	High

The AETHER system significantly improves efficiency and analytical capability.

9.11 Limitations Observed

Although the system performed effectively, certain limitations were identified:

- Dependence on external reconnaissance tools.
- Large scans increase resource usage.
- AI responses depend on available contextual data.
- Continuous updates required for emerging technologies.

These limitations provide direction for future enhancement.

9.12 Chapter Summary

The results demonstrate that the AETHER platform successfully automates reconnaissance, enhances analysis through artificial intelligence, and improves visualization of attack surfaces. The system meets its intended objectives and validates the effectiveness of the proposed architecture.

CHAPTER 10

CONCLUSION

The rapid expansion of web-based applications and digital services has significantly increased the complexity of modern cyber infrastructures, making traditional manual reconnaissance and analysis approaches insufficient. In response to this challenge, this project presented **AETHER**, an AI-assisted, modular, and legally compliant platform for automated web reconnaissance, attack surface mapping, and intelligent security analysis.

AETHER integrates focused web crawling, graph-based visualization, controlled execution of reconnaissance tools, and AI-powered reasoning to provide a unified view of an organization's external digital footprint. By structuring discovered assets into interactive graphs and storing reconnaissance outputs in a secure and scalable manner, the system enables security analysts to understand relationships between endpoints, services, and exposed resources more effectively. The inclusion of a Retrieval-Augmented Generation (RAG) based conversational interface further enhances usability by allowing users to query scan results and receive contextual, evidence-backed insights.

The proposed system emphasizes ethical usage, authorization validation, and non-exploitative analysis, ensuring compliance with legal and organizational security policies. Experimental results demonstrated that AETHER improves visibility, reduces manual effort, and offers better interpretability when compared to existing fragmented reconnaissance tools.

In conclusion, AETHER successfully addresses the limitations of existing reconnaissance methodologies by combining automation, visualization, and AI-driven intelligence into a single extensible platform. The system lays a strong foundation for future enhancements such as continuous monitoring, vulnerability prioritization, and predictive risk analytics, thereby contributing meaningfully to modern cybersecurity assessment practices.

REFERENCES

[1] M. G. Song, J. Choi, and H. K. Kim, "GAT-APG: Graph Attention Network-Based Attack Path Generation for Security Simulation," *IEEE Access*, vol. 13, pp. 147383–147396, 2025.

[2] H. Jmal, F. B. Hmida, N. Basta, M. Ikram, M. A. Kaafar, and A. Walker, "SPGNN-API: A Transferable Graph Neural Network for Attack Paths Identification and Autonomous Mitigation," *IEEE Trans. Inf. Forensics Secur.*, 2024.

[3] R. Barron, M. Eren, and C. Matuszek, "Cyber-Security Knowledge Graph Generation by Hierarchical Nonnegative Matrix Factorization," in *Proc. 2024 12th Int. Symp. Digital Forensics and Security (ISDFS)*, 2024.

[4] N. P. Kumar and V. K. Gowda K. K., "AIPenTool: A Unified Approach to Automated Penetration Testing for Enhanced Network and Web Application Security," in *Proc. 2025 IEEE Conf.*, 2025.

[5] Sahil G. K. and C. P. Chavan, "Automated Penetration Testing Tool," in *Proc. 2025 World Skills Conf. Universal Data Analytics and Sciences (WorldSUAS)*, 2025.

[6] P. Zhang, W. Wang, D. Zhang, and H. Wu, "Automated Security Penetration Testing Based on Machine Learning," in *Proc. 2024 4th Int. Conf. Electron. Inf. Eng. Comput. Commun. (EIECC)*, 2024.

[7] A. B. Samgir, V. Gutte, K. Kolhe, and D. R. Patil, "Automated Penetration Testing Architecture Using Metasploit and OWASP ZAP for Web Applications," in *Proc. 2024 2nd Int. Conf. Sustainable Comput. and Data Commun. Systems (ICSCDS)*, 2024.

[8] Y. Wang, Y. Li, X. Xiong, and C. Shen, "Automated Penetration Testing Based on Adversarial Inverse Reinforcement Learning," *IEEE Access*, vol. 12, 2024.

[9] V. Saber, D. ElSayad, A. Bahaa-Eldin, and Z. Fayed, "Automated Penetration Testing, A Systematic Review," in *Proc. 2023 Int. Mobile, Intell. Ubiquitous Comput. Conf. (MIUCC)*, 2023.

[10] D. B. Cruz, J. R. Almeida, and J. L. Oliveira, "Open Source Solutions for Vulnerability Assessment: A Comparative Analysis," *IEEE Access*, vol. 11, pp. 10325–10344, 2023.

[11] S. Yang, W. Pan, W. Zheng, and J. Xie, "Scorpio: an Automated Penetration Testing Tool and Its Integration with a Cyber Range," in *Proc. 2021 IEEE 4th Int. Conf. Electron. Inf. and Commun. Technol. (ICEICT)*, 2021.

[12] G. P. M. D. Silva et al., "So{U}RCERER: Developer-Driven Security Testing Framework for Android Apps," in *Proc. 2021 36th IEEE/ACM Int. Conf. Automated Softw. Eng. Workshops(ASEW)*, 2021.

[13] M. Malkawi and R. Alhajj, "Parallelized Cyber Reconnaissance Automation: A Real-Time and Scheduled Security Scanner," in *Cyber Security and Social Media Applications*, Springer, 2023.

[14] O. van Rooij, M. A. Charalambous, D. Kaizer, M. Papaevripides, and E. Athanasopoulos, "webFuzz: Grey-Box Fuzzing for Web Applications," in *Proc. Eur. Symp. Res. Comput. Secur. (ESORICS)*, 2021, pp. 152–172.

[15] G. Ban, L. Xu, Y. Xiao, X. Li, Z. Yuan, and W. Huo, "B2SMatcher: Fine-Grained Version Identification of Open-Source Software in Binary Files," *Cybersecurity*, vol. 4, no. 1, 2021.

[16] Y. Xu, Y. Liu, Z. Sun, Y. Xue, W. Liao, C. Liu, and Z. Sun, "Key Vulnerable Nodes Discovery Based on Bayesian Attack Subgraphs and Improved Fuzzy C-Means Clustering," *Mathematics*, vol. 12, no. 10, p. 1447, 2024.

[17] K. Abdulghaffar, N. Elmrabit, and M. Yousefi, "Enhancing Web Application Security through Automated Penetration Testing with Multiple Vulnerability Scanners," *Computers*, vol. 12, no. 11, p. 235, 2023.

[18] A. Pallakonda et al., "AI-Driven Attack Detection and Cryptographic Privacy Protection for Cyber-Resilient Industrial Control Systems," *IoT*, vol. 6, no. 3, 2025.

[19] I. Mutambik, "AI-Driven Cybersecurity in IoT: Adaptive Malware Detection and Lightweight Encryption via TRIM-SEC Framework," *Sensors*, vol. 25, no. 22, p. 7072, 2025.

[20] M. Kamande, K. Assa-Agyei, F. E. J. Broni, T. Al-Hadhrami, and I. Aqeel, "AI-Driven Threat Hunting in Enterprise Networks Using Hybrid CNN-LSTM Models for Anomaly Detection," *AI*, vol. 6, no. 12, p. 306, 2025.

[21] N. Rane and A. Qureshi, "Comparative Analysis of Automated Scanning and Manual Penetration Testing for Enhanced Cybersecurity," in *Proc. 12th Int. Symp. Digital Forensics and Security (ISDFS)*, 2024.

[22] U. D. Ani, J. Watson, H. He, P. Radanliev, and G. Epiphaniou, "Minimising Cybersecurity Risk Exposures in Industrial Control System Environments: ATechno-Human Vulnerability Analysis Approach," *Int. J. Syst. Assur. Eng. Manag.*, 2024.

[23] E. Islam, C. Rudolph, and G. Oliver, "Managing Cyber Harm: A Survey of Challenges, Practices, and Opportunities," *Inf. Secur. J.: A Global Perspect.*, vol. 34, no. 5, pp. 424–454, 2025.

[24] C. Park, C. Kontovas, Z. Yang, and C.-H. Chang, "Building Cyber Resilience in Maritime Transport: A Stakeholders' Perspective on Mitigation Measures," *J. Int. Maritime Safety, Environ. Affairs, and Shipping*, vol. 9, no. 3-4, 2025.

[25] K. Kurniawan, R. F. Ardian, E. Kiesling, and A. Ekelhart, "AgCyRAG: an Agentic Knowledge Graph based RAG Framework for Automated Security Analysis," in *Proc. CEUR Workshop*, 2025

JOURNAL/IEEE CONFERENCE PUBLICATIONS

This project is suitable for submission to:

- IEEE International Conference on Cyber Security and Resilience
- IEEE Conference on Trust, Security and Privacy
- ACM CCS Student Research Competition
- Springer Journal of Cybersecurity

APPENDIX

APPENDIX – A

SYSTEM SCREENSHOTS

A.2 Dashboard Interface

Description: Central dashboard displaying projects, scan status, and analytics overview.

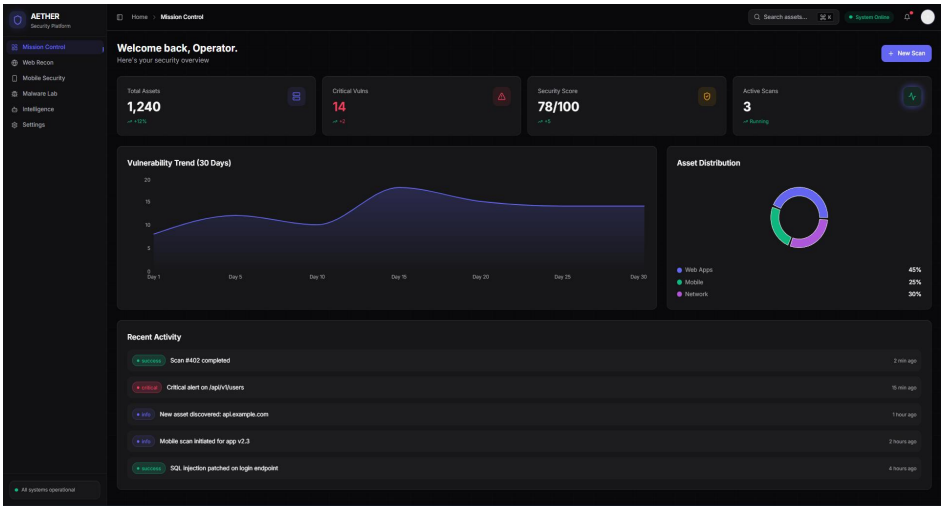

Figure A.2: System Dashboard

A.3 Project Creation Module

Description: Interface used to define authorized targets and reconnaissance scope.

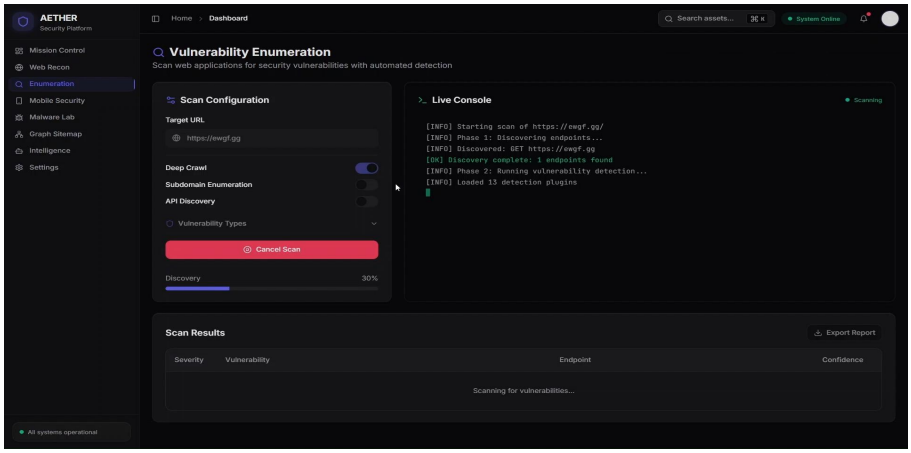

Figure A.3: Project Creation Screen

A.4 Reconnaissance Execution

Description: Shows initiation of automated reconnaissance scan.

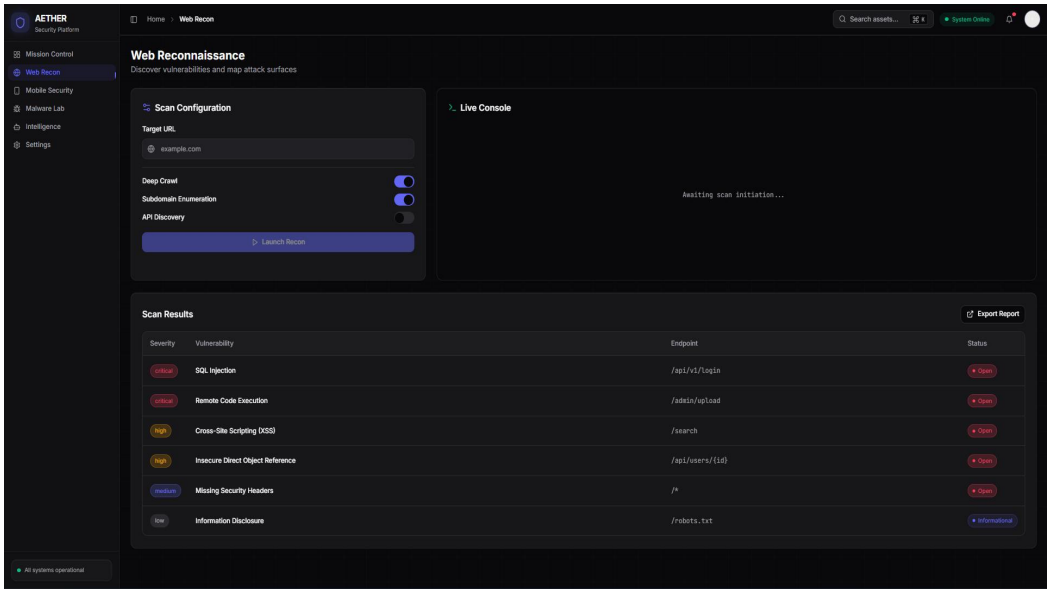

Figure A.4: Recon Scan Execution

A.5 Scan Results View

Description: Displays discovered assets including domains, endpoints, and technologies.

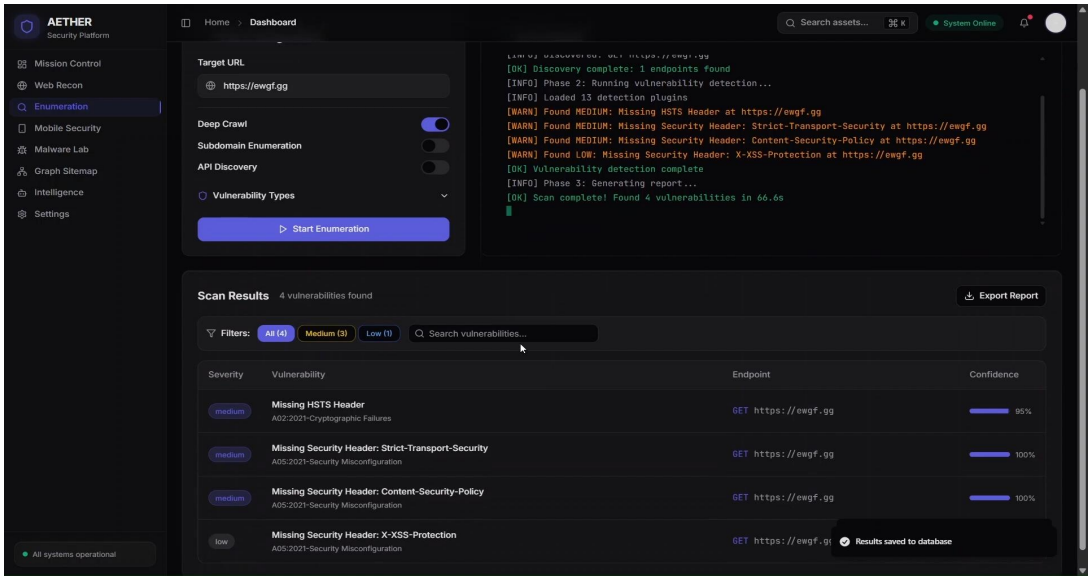

Figure A.5: Recon Results Interface

A.6 Graph Visualization Module

Description: Interactive graph representation of attack surface relationships.

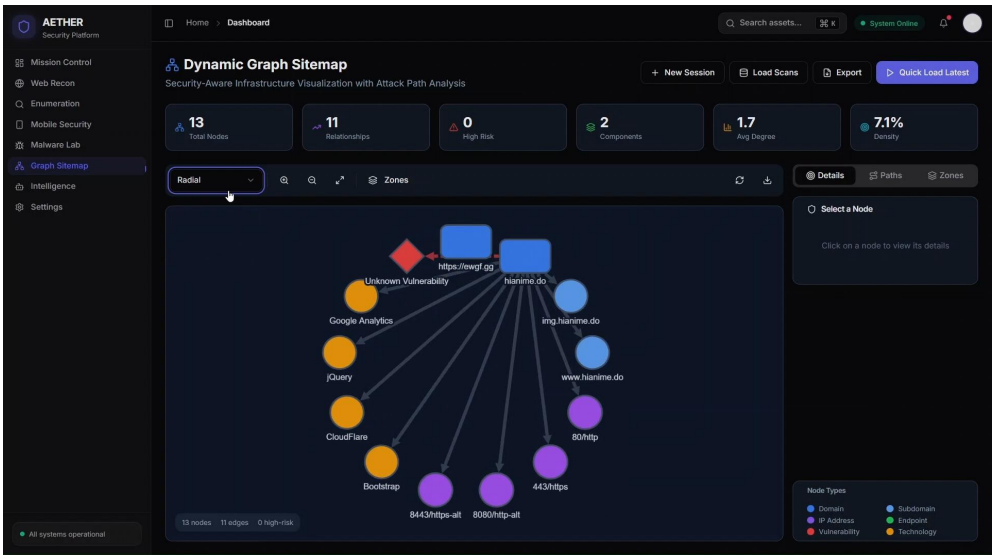

Figure A.6: Attack Surface Graph Visualization (Insert Screenshot)

A.7 AI Assistant Interface

Description: Natural language interaction with AI intelligence module.

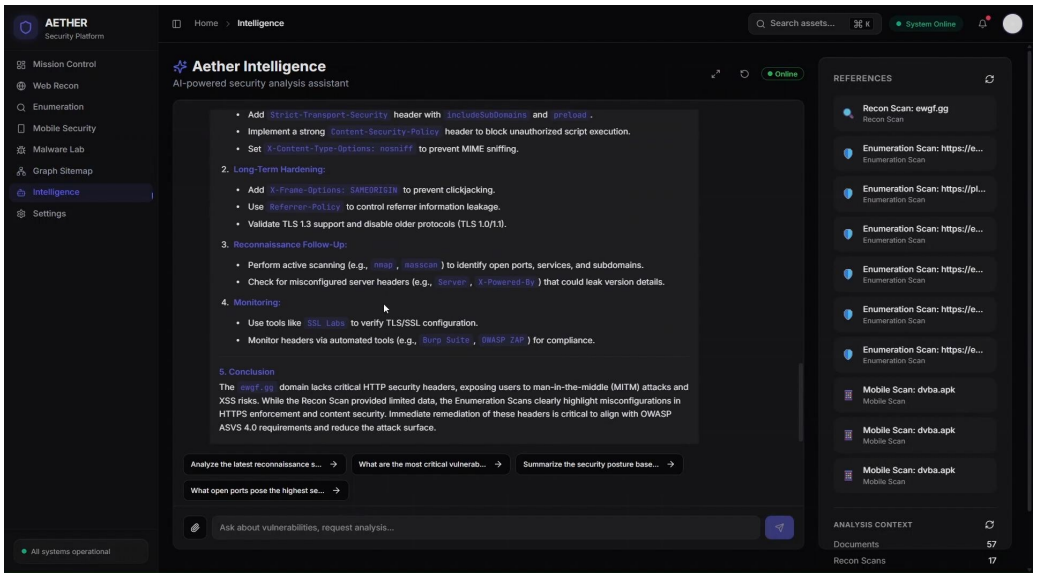

Figure A.7: AI Query System (Insert Screenshot)

A.8 Mobile Security Analysis

Description: Static analysis report generated for uploaded APK file.

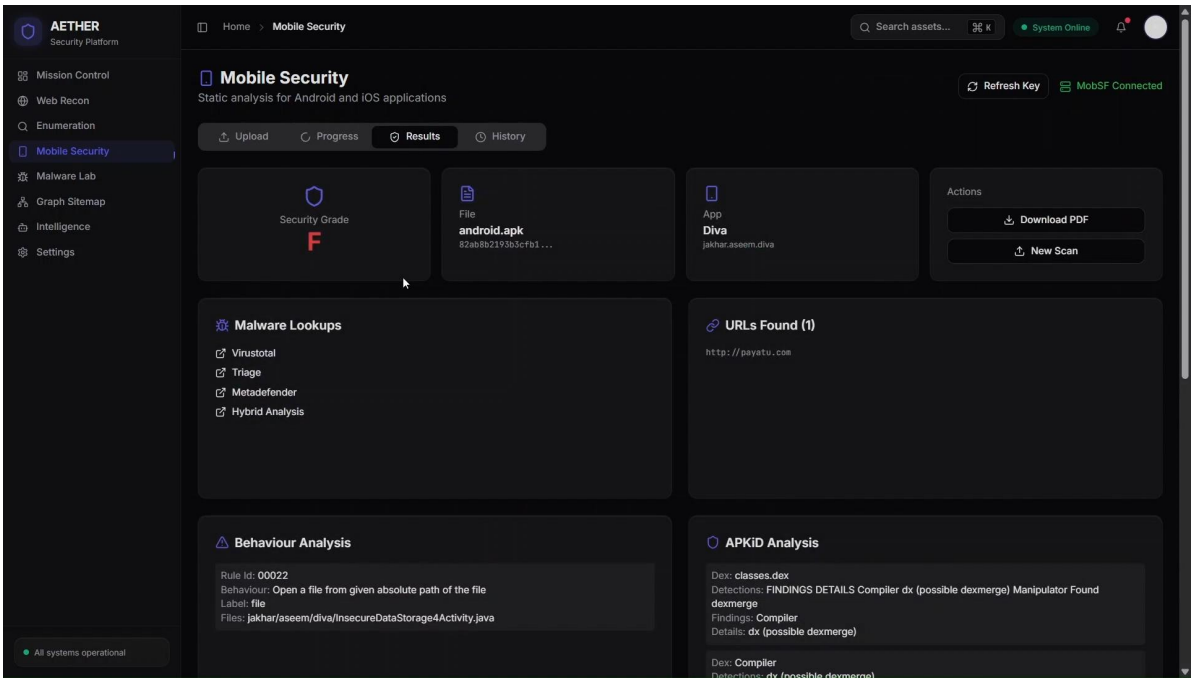

Figure A.8: Mobile Analysis Report

APPENDIX – B

SYSTEM API DOCUMENTATION

B.1 API Architecture Overview

The AETHER system uses RESTful APIs for communication between frontend, backend, and worker services. All APIs exchange data using JSON format.

B.2 Authentication APIs

API	Method	Description
/api/auth/register	POST	Register new user
/api/auth/login	POST	User authentication
/api/auth/profile	GET	Retrieve user profile
/api/auth/logout	POST	Logout session

B.3 Project Management APIs

API	Method	Description
/api/projects	POST	Create project
/api/projects	GET	List projects
/api/projects/{id}	GET	View project
/api/projects/{id}	DELETE	Delete project

B.4 Reconnaissance APIs

API	Method	Description
/api/recon/start	POST	Start scan
/api/recon/status/{id}	GET	Scan status
/api/recon/results/{id}	GET	Retrieve results

B.5 Crawling APIs

API	Method	Description
/api/crawler/start	POST	Start crawler
/api/crawler/results/{id}	GET	Crawl output

B.6 AI Intelligence APIs

API	Method	Description
/api/ai/query	POST	Ask AI question
/api/ai/summarize	POST	Generate summary

B.7 Job Queue APIs

API	Method	Description
/api/jobs/create	POST	Create job
/api/jobs/status/{id}	GET	Job status

APPENDIX – C

TECHNOLOGY STACK

C.1 Programming Languages

- Python
- JavaScript
- HTML
- CSS

C.2 Frontend Technologies

- React.js
- Tailwind CSS
- Axios
- Dynamic Component Rendering

C.3 Backend Technologies

- FastAPI Framework
- REST API Architecture
- Async Programming

C.4 Database Technologies

- PostgreSQL (Relational Database)
- Vector Database (AI Embeddings Storage)

C.5 AI Technologies

- Retrieval-Augmented Generation (RAG)
- Embedding Models
- Natural Language Processing

C.6 DevOps & Deployment

- Docker Containers
- Redis Queue System
- Git Version Control
- Linux Environment

APPENDIX – D

SYSTEM ARCHITECTURE SNAPSHOT

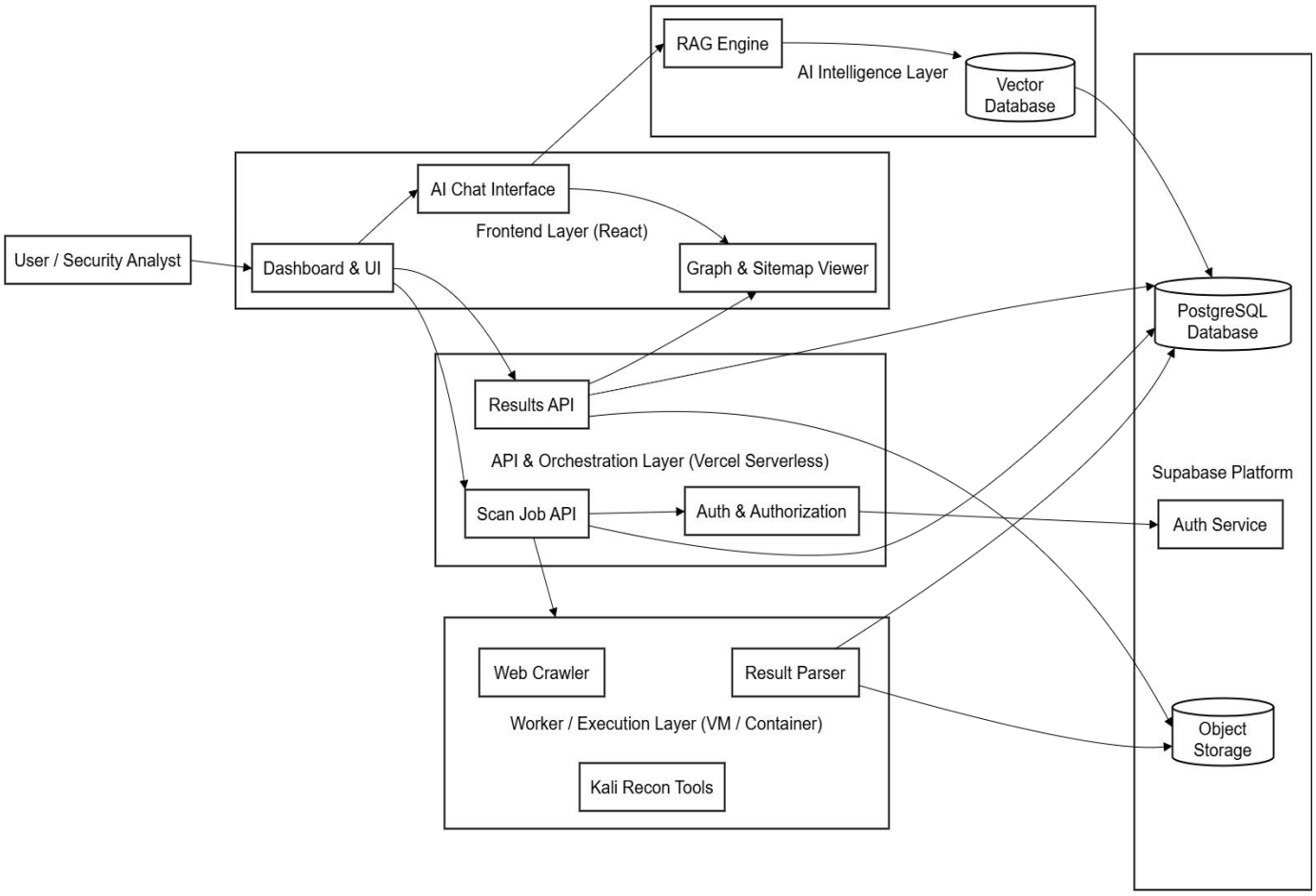

Figure D.1: Overall System Architecture Diagram

Description:

- User interacts with frontend.
- Backend APIs process requests.
- Worker nodes execute scans.
- Database stores results.
- AI module analyzes data.

APPENDIX – E

DATABASE SCHEMA

E.1 Users Table

Field	Type
id	Integer
email	String
password_hash	String
role	String

E.2 Projects Table

Field	Type
project_id	Integer
target_domain	String
scope	Text
created_at	Timestamp

E.3 Recon Results Table

Field	Type
result_id	Integer
project_id	Integer
asset_type	String
data	JSON

APPENDIX – F

SAMPLE OUTPUTS

F.1 Sample Recon Result

```
Target Domain: example.com
Subdomains Found: 25
Technologies Detected: Nginx, React
Endpoints Discovered: 112
```

F.2 Sample AI Query

```
User Query:
Explain major risks.

AI Response:
Exposed API endpoints may allow unauthorized access.
```

APPENDIX – G

SOURCE CODE STRUCTURE

```
AETHER/
│
├── frontend/
├── backend/
│   ├── auth/
│   ├── projects/
│   ├── recon/
│   ├── ai/
│   ├── crawler/
│   └── workers/
│
│
├── database/
├── docker/
└── docs/
```

APPENDIX – H

DEPLOYMENT DETAILS

Deployment Steps

1. Install Docker.
2. Configure environment variables.
3. Start PostgreSQL and Redis.
4. Launch backend server.
5. Start worker services.
6. Deploy frontend application.